

การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์

Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence

ชื่อผู้จัดทำ

นายจิรายุ สงบวาจา

นางสาวอรไพลิน สอาดเอี่ยม

นางสาววนิดา คล้ายสกุล

โครงงานวิจัยระดับปริญญาตรีนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ พ.ศ. 2569

การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์

Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence

ชื่อผู้จัดทำ

นายจิรายุ สงบวาจา

นางสาวอรไพลิน สอาดเอี่ยม

นางสาววนิดา คล้ายสกุล

โครงงานวิจัยระดับปริญญาตรีนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ พ.ศ. 2569

หัวข้อโครงงานวิจัย	การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์ Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence
ชื่อนักศึกษา	นายจิรายุ สงบวาจา นางสาวอรไพลิน สอาดเอี่ยม นางสาววนิดา คล้ายสกุล
อาจารย์ที่ปรึกษา	อาจารย์ธนวัฒน์ ถาวรกูล ผศ.วราภรณ์ มั่นทุ่ง
ระดับการศึกษา	ปริญญาตรี
สาขาวิชา	วิทยาการคอมพิวเตอร์
ปีการศึกษา	2569

บทคัดย่อ

โครงงานวิจัยนี้มีวัตถุประสงค์เพื่อศึกษาและพัฒนาเกมคอมพิวเตอร์สองมิติแนวเอาชีวิตรอดและลอบเร้น (Survival Stealth Game) โดยมุ่งเน้นการประยุกต์ใช้เทคนิคเครื่องสถานะจำกัด (Finite State Machine: FSM) ในการควบคุมสถานะของตัวละครผู้เล่นและระบบปัญญาประดิษฐ์ (Artificial Intelligence: AI) ภายในเกม เพื่อให้การทำงานของระบบมีโครงสร้างที่ชัดเจน ลดความซับซ้อนของโปรแกรม และเพิ่มประสิทธิภาพในการควบคุมพฤติกรรมของตัวละคร การพัฒนาเกมใช้โปรแกรม Unity Game Engine และเขียนโปรแกรมด้วยภาษา C# โดยนำแนวคิด FSM มาใช้เป็นโครงสร้างหลักในการจัดการสถานะของตัวละครผู้เล่น ได้แก่ การยืน การเดิน การนั่งย่อ และการหมอบคลาน ซึ่งแต่ละสถานะมีผลต่อพารามิเตอร์ภายในเกม เช่น ความเร็วในการเคลื่อนที่ ขนาดพื้นที่การชน (Collider) และระดับความดังของเสียงแบบเรียลไทม์ นอกจากนี้ได้พัฒนาระบบปัญญาประดิษฐ์ของตัวละครศัตรูให้สามารถตรวจจับผู้เล่นผ่านระบบการมองเห็นด้วยเทคนิค Raycasting และตอบสนองต่อระดับเสียงที่ผู้เล่นสร้างขึ้น.โดยทำงานร่วมกับอัลกอริทึมค้นหาเส้นทางแบบ A* (A* Pathfinding) เพื่อให้ศัตรูสามารถเคลื่อนที่หลบหลีกสิ่งกีดขวางและไล่ล่าผู้เล่นได้อย่างสมจริง ผลการดำเนินงานพบว่า การประยุกต์ใช้เทคนิค Finite State Machine ช่วยให้สามารถจัดการสถานะของตัวละครและเงื่อนไขการทำงานของระบบ AI ได้อย่างเป็นระบบ ลดความซับซ้อนของโค้ด และเพิ่มความยืดหยุ่นในการพัฒนา ซอฟต์แวร์ต้นแบบที่พัฒนาขึ้นสามารถตอบสนองต่อคำสั่งของผู้เล่นได้อย่างถูกต้อง และตัวละครศัตรูมีพฤติกรรมการไล่ล่าที่สอดคล้อง

กับตรรกะที่กำหนดไว้ บรรลุตามวัตถุประสงค์ของโครงงานในการประยุกต์ใช้ความรู้ด้านวิทยาการคอมพิวเตอร์เพื่อพัฒนาสื่อบันเทิงเชิงโต้ตอบ

กิตติกรรมประกาศ

โครงงานวิจัยฉบับนี้สำเร็จลุล่วงไปได้ด้วยดีเนื่องด้วยความกรุณาจาก อาจารย์ธนวัฒน์ ถาวรกูล และ ผศ.วราภรณ์ มั่นทุ่ง อาจารย์ที่ปรึกษาโครงงาน ที่ได้ให้คำปรึกษา แนะนำแนวทาง และแก้ไขข้อบกพร่องต่าง ๆ ในการพัฒนาเกมสยองขวัญ 2 มิติเชิงจริยธรรมด้วยภาษา C# ตลอดจนถ่ายทอดความรู้และเทคนิคต่าง ๆ ในการใช้โปรแกรม Unity จนโครงงานเล่มนี้เสร็จสมบูรณ์

ขอกราบขอบพระคุณบิดา มารดา และครอบครัว ที่ให้การสนับสนุนทั้งในด้านกำลังใจและทุนทรัพย์ในการศึกษาเล่าเรียนเสมอมา รวมถึงขอขอบคุณเพื่อนนักศึกษาทุกท่านที่ให้ความช่วยเหลือในการทดสอบระบบเกมและให้คำแนะนำที่เป็นประโยชน์ต่อการพัฒนาโครงงานในครั้งนี้

คุณค่าและประโยชน์ของโครงงานวิจัยฉบับนี้ ผู้จัดทำขอมอบเป็นเครื่องบูชาพระคุณบิดา มารดา ครูบาอาจารย์ และผู้มีพระคุณทุกท่านที่มีส่วนช่วยให้ผู้จัดทำประสบความสำเร็จในการศึกษา

ชื่อผู้จัดทำ

จิรายุ สงบวาจา

อรไพลิน สอาดเอี่ยม

วนิดา คล้ายสกุล

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

การพัฒนาเกมคอมพิวเตอร์ในปัจจุบันมีความซับซ้อนทางด้านวิศวกรรมซอฟต์แวร์มากขึ้น โดยเฉพาะในเกมแนวลอบเร้น (Stealth Survival) หัวใจสำคัญไม่ได้อยู่ที่ความสวยงามเพียงอย่างเดียว แต่อยู่ที่ "การออกแบบโครงสร้างโค้ด (Software Architecture)" เพื่อควบคุมสถานะของตัวละคร (Player State Management) ที่มีความละเอียดซับซ้อน เช่น การเปลี่ยนอิริยาบถจากการยืน เป็นการนั่งย่อ หรือการหมอบคลาน ซึ่งการกระทำเหล่านี้ต้องส่งผลต่อค่าพารามิเตอร์ต่างๆ ทางฟิสิกส์และการตรวจจับภายในเกมแบบเรียลไทม์

นอกจากนี้ การสร้างศัตรูคอมพิวเตอร์ (AI Agent) ให้มีความฉลาดและตอบสนองต่อพฤติกรรมของผู้เล่นอย่างสมจริง จำเป็นต้องประยุกต์ใช้อัลกอริทึมขั้นสูง เช่น Finite State Machine (FSM) ในการจำแนกสถานะ และการคำนวณทางคณิตศาสตร์เวกเตอร์ (Vector Mathematics) เพื่อตรวจสอบการมองเห็น (Line of Sight) และการได้ยินเสียงที่แปรผันตามท่าทางของผู้เล่น

ผู้จัดทำจึงมีแนวคิดพัฒนาโครงงาน "การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์" โดยใช้ Unity Game Engine และภาษา C# โครงงานนี้จะมุ่งเน้นการเขียนโปรแกรมเพื่อจัดการกลไกการเคลื่อนไหวที่ซับซ้อน (Advanced Movement Mechanics) และระบบ AI ที่ไล่ล่าผู้เล่นอย่างเป็นระบบ เพื่อเป็นกรณีศึกษาการนำความรู้วิทยาการคอมพิวเตอร์มาประยุกต์ใช้ในการสร้างซอฟต์แวร์ที่มีการโต้ตอบสูง

1.2 วัตถุประสงค์ของโครงงาน

1.2.1 เพื่อพัฒนาเกมคอมพิวเตอร์ 2 มิติ ด้วยภาษา C# โดยเน้นการวางโครงสร้างโค้ดแบบเชิงวัตถุ (Object-Oriented Programming)

1.2.2 เพื่อประยุกต์ใช้โครงสร้างข้อมูลแบบ Finite State Machine (FSM) ในการจัดการสถานะการเคลื่อนไหวของตัวละครผู้เล่น (Movement States

1.2.3 เพื่อพัฒนาอัลกอริทึมปัญญาประดิษฐ์ (AI) สำหรับศัตรูที่สามารถตอบสนองต่อระดับเสียงและขนาดพื้นที่การมองเห็น (Hitbox) ที่เปลี่ยนแปลงไปตามสถานะของผู้เล่นได้

1.2.4 เพื่อสร้างระบบการเล่น (Gameplay Loop) ที่สมบูรณ์ ทั้งการเก็บไอเทม การแก้ไขปริศนา และการเปลี่ยนฉากเมื่อผ่านเงื่อนไข

1.2.5 เพื่อประเมินความพึงพอใจของผู้เล่นต่อเกมที่พัฒนาขึ้น เพื่อนำไปปรับปรุงและพัฒนาระบบให้มีประสิทธิภาพยิ่งขึ้น

1.3 ขอบเขตของโครงงาน

1.3.1 ส่วนพัฒนาระบบควบคุมตัวละคร (Player Controller System)

พัฒนาระบบควบคุมตัวละครผู้เล่นให้สามารถเคลื่อนไหวได้หลากหลายรูปแบบ และส่งผลต่อการเล่นในเชิงกลยุทธ์ เช่น การลดเสียงเพื่อหลบศัตรู

1.3.1.1 State Management

พัฒนาระบบสถานะของตัวละครโดยใช้ Finite State Machine (FSM) เพื่อควบคุมการเปลี่ยนสถานะระหว่าง Idle, Walk, Crouch และ Prone อย่างเป็นลำดับ

1.3.1.2 Dynamic Collider Adjustment

พัฒนาระบบปรับขนาด Hitbox ตามท่าทางของตัวละครแบบเรียลไทม์ เพื่อรองรับการเคลื่อนที่ในพื้นที่จำกัด เช่น การหมอบลอดช่องแคบ

1.3.1.3 Variable Parameters

กำหนดค่าความเร็วและระดับเสียงในแต่ละสถานะ เพื่อใช้เป็นข้อมูลในการตรวจจับของศัตรู

1.3.2 ส่วนพัฒนาระบบปัญญาประดิษฐ์ของศัตรู (Enemy AI System)

พัฒนาระบบ AI ที่สามารถตอบสนองต่อพฤติกรรมของผู้เล่น โดยเน้นการตรวจจับและการตัดสินใจแบบเรียลไทม์

1.3.2.1 Behavior System (FSM)

กำหนดสถานะของศัตรู เช่น Patrol, Investigate, Chaseและ Attack เพื่อควบคุมพฤติกรรม

1.3.2.2 Sensory System

พัฒนาระบบการมองเห็นด้วย Raycasting และระบบการได้ยินโดยใช้การคำนวณระยะทางร่วมกับระดับเสียงของผู้เล่น

1.3.2.3 Pathfinding

ใช้ Navigation Mesh และ A* Algorithm เพื่อให้ศัตรูสามารถเคลื่อนที่หลบสิ่งกีดขวางและติดตามผู้เล่นได้

1.3.3 ส่วนพัฒนาระบบเกม (Game Logic System)

พัฒนากลไกหลักของเกมเพื่อกำหนดเป้าหมายและลำดับการเล่น

1.3.3.1 Inventory และ Mission Logic

ผู้เล่นต้องเก็บไอเทมให้ครบจำนวนที่กำหนด (เช่น 5 ชิ้น) เพื่อปลดล็อกทางออก

1.3.3.2 Scene Management

รองรับการเปลี่ยนฉากและการเก็บข้อมูลความคืบหน้าของเกม

1.3.4 ส่วนออกแบบส่วนติดต่อผู้ใช้งาน (UI/UX)

เป็นการออกแบบส่วนติดต่อผู้ใช้เพื่อแสดงข้อมูลสำคัญ เช่น สถานะตัวละคร ระดับเสียง และจำนวนไอเทม เพื่อช่วยให้ผู้เล่นสามารถวางแผนการเล่นและตัดสินใจได้อย่างมีประสิทธิภาพ

1.3.5 ขอบเขตด้านเนื้อหาเกม (Game Content Scope)

1.3.5.1 เกมที่พัฒนาเป็นเกมคอมพิวเตอร์แบบ 2 มิติ แนวลอบเร้น (Stealth Gameโดยผู้เล่นต้องหลบหลีกศัตรูและวางแผนการเคลื่อนที่

1.3.5.2 ภายในเกมมีฉากการเล่นอย่างน้อย 1–2 ฉาก ซึ่งออกแบบให้มีสิ่งกีดขวางและพื้นที่สำหรับการหลบซ่อน

1.3.5.3 มีตัวละครศัตรูอย่างน้อย 1 ประเภท ที่สามารถลาดตระเวน ตรวจจับเสียง และไล่ล่าผู้เล่นได้

1.3.5.4 มีระบบการเก็บไอเทม โดยผู้เล่นต้องรวบรวมไอเทมให้ครบตามจำนวนที่กำหนดเพื่อผ่านด่าน

1.3.5.5 มีเงื่อนไขการชนะ (Win Condition) เมื่อผู้เล่นเก็บไอเทมครบและไปถึงทางออกของฉาก

1.3.5.6 มีเงื่อนไขการแพ้ (Lose Condition) เมื่อผู้เล่นถูกศัตรูตรวจจับและพลังชีวิหมด

1.3.6 ขอบเขตด้านเทคโนโลยี (Technology Scope)

1.3.6.1 ใช้ Unity Game Engine ในการพัฒนาเกม

1.3.6.2 ใช้ภาษา C# ในการเขียนโปรแกรมควบคุมระบบต่าง ๆ ภายในเกม

1.3.6.3 ใช้แนวคิดการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming OOP) ในการออกแบบโครงสร้างระบบ

1.3.6.4 ใช้เทคนิค Finite State Machine (FSM) ในการควบคุมพฤติกรรมของตัละคผู้เล่นและศัตรู

1.3.6.5 ใช้ Raycasting ในการพัฒนาระบบการมองเห็นของศัตรู และใช้การคำนวณระยะทางสำหรับระบบตรวจจับเสียง

1.3.6.6 ใช้ Navigation Mesh ร่วมกับอัลกอริทึม A* (A-Star Algorithm) ในการค้นหาเส้นทางของศัตรู

1.4 เครื่องมือที่เกี่ยวข้องกับโครงงาน

1.4.1 Unity Game Engine เครื่องมือหลักในการพัฒนา

1.4.2 ภาษา C# สำหรับเขียนสคริปต์ควบคุมการทำงาน

1.4.3 Visual Studio: IDE สำหรับเขียนโค้ดและ Debug

1.4.4 โปรแกรมสร้างกราฟิก 2D: สำหรับสร้าง Sprite ตัวละครและฉาก

1.5 ตารางการดำเนินโครงงาน

การดำเนินโครงงานวิจัย	ระยะเวลาการดำเนินโครงงานวิจัย																							
	พ.ย.		ธ.ค.		ม.ค.		ก.พ.		มี.ค.		เม.ย.		พ.ค.		มิ.ย.		ก.ค.		ส.ค.		ก.ย.	
1. ศึกษาปัญหาและความต้องการ																						
2. กำหนดขอบเขต																						
3. ศึกษาการใช้งาน Unity																						
4. ออกแบบตัวละคร																						
5. ออกแบบระบบ																						
6. พัฒนาระบบ																						
7. ติดตั้งโปรแกรมและทดสอบ																						
8. ปรับปรุงและแก้ไขระบบ																						
10. ทำเอกสารงานวิจัย																						
11. พบอาจารย์ที่ปรึกษา																						

ระยะเวลาคาดหวัง

ระยะเวลาทำงานจริง

1.6 ประโยชน์ที่คาดว่าจะได้รับ

1.6.1 ได้รับความรู้ความเข้าใจเชิงลึกในการเขียนโปรแกรมจัดการสถานะ (State Management) ซึ่งเป็นพื้นฐานสำคัญของการพัฒนาซอฟต์แวร์

1.6.2 สามารถประยุกต์ใช้คณิตศาสตร์และอัลกอริทึม (Physics & Algorithms) ในการสร้างระบบการตรวจจับ (Detection System) ภายในเกมได้

1.6.3 ได้ต้นแบบซอฟต์แวร์เกมที่มีระบบการควบคุมตัวละคร (Character Controller) และ AI ที่ทำงานสัมพันธ์กันอย่างเป็นระบบ

1.6.4 มีทักษะในการออกแบบ UX/UI ที่สื่อสารข้อมูลเชิงเทคนิค (เช่น ระดับเสียง) ให้ผู้ใช้งานเข้าใจได้ง่าย

1.6.5 ทำให้ทราบระดับความพึงพอใจของผู้เล่น และสามารถนำผลที่ได้ไปใช้ในการปรับปรุงและพัฒนาระบบเกมให้มีประสิทธิภาพและตอบสนองต่อผู้ใช้งานได้ดียิ่งขึ้น

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

ในการพัฒนาเกมคอมพิวเตอร์ 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game ด้วย C# ผู้จัดทำได้ศึกษาหลักการทฤษฎีและเทคโนโลยีที่ เกี่ยวข้อง เพื่อใช้เป็นแนวทางในการดำเนินงานวิจัย ประกอบไปด้วยหัวข้อหลัก ๆ ดังต่อไปนี้

2.1 แนวคิดเกี่ยวกับการพัฒนาเกม (Game Development)

2.2 การพัฒนาเกม 2 มิติ

2.3 เกม Side-scrolling game หรือ Side-view game

2.4 เกมประเภท Hide-and-Seek Horror Game

2.5 แนวคิดเกี่ยวกับ Finite State Machine (FSM)

2.6 การพัฒนาปัญญาประดิษฐ์ในเกม

2.7 ภาษาและเครื่องมือที่ใช้พัฒนา

2.8 อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm)

2.9 การออกแบบ ux ui ให้เหมาะสม

2.10 งานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดเกี่ยวกับการพัฒนาเกม (Game Development)

การพัฒนาเกม (Game Development) เป็นกระบวนการสร้างซอฟต์แวร์ เชิงโต้ตอบที่ผสานองค์ประกอบด้านการออกแบบ ศิลปะ และวิศวกรรมซอฟต์แวร์เข้าด้วย กัน โดยทั่วไปกระบวนการพัฒนาเกมประกอบด้วยขั้นตอนการวิเคราะห์ความต้องการ (Requirement Analysis) การออกแบบระบบ (System Design) การพัฒนา (Implementation) การทดสอบ (Testing) และการประเมินผล (Evaluation) กระบวนการสร้างเกม (Game Development) คือ กระบวนการในมุมมอง ของการสร้าง การออกแบบ โดยใช้ประกอบในมุมมองสองมิติ โดยการออกประกอบจาก เกม X (แนวนอน) และ แนวนอน Y (แนวตั้ง) ซึ่งแตกต่างจากเกม 3 มิติที่สามารถแสดง ตัวละครจากเกม ในการรวมกันสร้างสรรค์เกมในการออกแบบ เกม 2 มิติจะดำเนินการใน พื้นที่ หรือสิ่งต่างๆ ในการออกแบบ ทั้งนี้ เกม 2 มิติที่มีสามารถแสดงสถานการณ์สำหรับผู้ เล่นในเนื้อหาหรืออาจจะใช้ในการออกแบบแนวเกมสนุก ที่ตัวละครจะสร้างสรรค์กลยุทธ์ และสามารถสร้างความประทับใจให้ผู้เล่นได้ องค์ประกอบสำคัญของเกม ได้แก่ ระบบกลไกการเล่น (Game Mechanics) ระบบกราฟิกและแอนิเมชัน ระบบเสียง ระบบปัญญาประดิษฐ์ (AI System) และส่วน ติดต่อผู้ใช้ (User Interface) ในเกมที่มีศัตรูหรือ NPC ระบบ AI มีบทบาทสำคัญใน

การสร้างความท้าทายและความสมจริง โดยเฉพาะเกมแนวสยองขวัญ (Horror Game) ที่พฤติกรรมของศัตรูต้องตอบสนองต่อการกระทำของผู้เล่นอย่างมีเหตุผล

2.2 การพัฒนาเกม 2 มิติ

การพัฒนาเกม 2 มิติ (2D Game Development) เป็นการสร้างเกมที่ใช้ พิกัดสองมิติในการแสดงผล ประกอบด้วยแกน X (แนวนอน) และแกน Y (แนวตั้ง) ซึ่ง แตกต่างจากเกม 3 มิติที่มีความลึก (Depth) เพิ่มเติม เกม 2 มิติมีความได้เปรียบใน ด้านความง่ายต่อการพัฒนา สุนทรียศาสตร์ที่มีเอกลักษณ์ได้ ใช้ทรัพยากรการประมวลผลต่ำกว่า และสามารถสร้าง นักพัฒนาจะต้องออกแบบและควบคุมสภาพแวดล้อม โดยระบบดำเนินการเส้นทาง (Platformer) ร่วมลักษณะการทำงานในสามารถทำงานได้ดีในเกม 2 มิติ โดยแตก ต่างจากเกม 3 มิติในด้านความซับซ้อน ในการพัฒนาเกม 2 มิติ เครื่องมือที่นิยมใช้คือUnity Game Engine ซึ่งรองรับการพัฒนาเกม 2 มิติได้อย่างครบถ้วน ทั้งระบบ Sprite, Tilemaps, Physics 2D และ Animator สำหรับการสร้างแอนิเมชั่นตัว ละคร ส่วนสำคัญที่สำคัญในการพัฒนาเกม 2 มิติ คือ การออกแบบระดับ (Level Design) ซึ่งเป็นการเลือกรูปแบบและทรัพยากรในการพัฒนาเกม รวมถึงการออกแบบ Tilemap เพื่อสร้างฉากที่มีความสวยงามและน่าสนใจ นอกจากนี้ระบบ Collision Detection ก็มีความสำคัญในการตรวจจับการชนระหว่างตัวละครกับสิ่งแวดล้อม ภายในเกม

2.3 เกม Side-scrolling game หรือ Side-view game

เกม Side-scrolling game หรือ Side-view game คือเกมที่มุมกล้อง ถูกกำหนดให้มองจากด้านข้างของตัวละคร ทำให้ผู้เล่นเห็นตัวละครในแนวตั้งและสามารถ เคลื่อนที่ไปทางซ้ายหรือขวาตามแกน X เป็นหลัก มุมมองแบบนี้เป็นที่นิยมอย่างกว้างขวางในเกมยุคแรก ๆ และยังคงได้รับความนิยมอย่างต่อเนื่องในเกมยุคปัจจุบัน เนื่องจาก สามารถสร้างประสบการณ์การเล่นที่ชัดเจนและเข้าใจง่าย คุณลักษณะสำคัญของเกม Side-scrolling ได้แก่ การเคลื่อนที่ของตัวละครใน แนวแกน X เป็นหลัก กล้องจะเลื่อนตาม (Follow Camera) ไปพร้อมกับตัวละคร การออกแบบฉากในแนวนอนที่มีความต่อเนื่อง รวมถึงการใช้ระบบชั้น (Layering) เพื่อสร้างความลึกในภาพแบบ 2 มิติ เช่น ฉากหลังเลื่อนช้ากว่าฉากหน้า (Parallax Scrolling) ซึ่งช่วยเพิ่มความสมจริงให้กับเกม ในการพัฒนาเกม Side-scrolling Horror Game นั้น มุมมองด้านข้างช่วย สร้างบรรยากาศความหวาดกลัวได้อย่างมีประสิทธิภาพ เนื่องจากผู้เล่นจะมองเห็นสิ่งที่อยู่ ข้างหน้าและข้างหลังได้อย่างชัดเจน แต่ไม่สามารถมองเห็นสิ่งที่อยู่ด้านบนหรือด้านล่างได้ ครบถ้วน ทำให้เกิดความตื่นเต้นและความไม่แน่นอนขณะเล่น

2.4 เกมประเภท Hide-and-Seek Horror Game

เกมประเภท Hide-and-Seek Horror Game คือเกมสยองขวัญที่มีแกน หลักของการเล่น (Core Gameplay) อยู่ที่การหลบซ่อนตัวจากศัตรูหรือผู้ไล่ล่า (Pursuer) ผู้เล่นจะต้องใช้ไหวพริบและความระมัดระวังในการหลบหลีกศัตรู รวบรวม ไอเทมที่กำหนด และหาทางออกจากด่านให้สำเร็จ โดยไม่ถูกศัตรูจับได้ เกมประเภทนี้มีชื่อเสียงเช่น Slender: The Eight Pages, Outlast, และ Amnesia: The Dark Descent องค์ประกอบสำคัญของ Hide-and-Seek Horror Game ได้แก่ ระบบ ตรวจจับของศัตรู (Enemy Detection System) ซึ่งประกอบด้วยการมองเห็น (Line of Sight) และการได้ยินเสียง (Sound Detection) ผู้เล่นจะต้องควบคุม ระดับเสียงที่เกิดจากการเคลื่อนไหว เช่น การเดินธรรมดาจะมีเสียงดังกว่าการนั่งย่อหรือหมอบคลาน รวมถึงการใช้สภาพแวดล้อม เช่น การซ่อนตัวในตู้หรือใต้โต๊ะ เพื่อหลบเลี่ยง การตรวจจับในเกม 2 มิติแนว Side-scrolling Horror Game การออกแบบระบบ Al ของศัตรูมีความสำคัญอย่างยิ่ง ศัตรูจะต้องมีพฤติกรรมที่สมเหตุสมผลและน่าเชื่อถือ โดย อาศัยเทคนิค Finite State Machine (FSM) ในการจัดการสถานะพฤติกรรมต่าง ๆ เช่น การลาดตระเวน การตรวจสอบเสียง และการไล่ล่า รวมถึงระบบ Raycasting สำหรับตรวจสอบการมองเห็น และอัลกอริทึม A* สำหรับการนำทางในฉาก

2.5 แนวคิดเกี่ยวกับ Finite State Machine (FSM)

Finite State Machine (FSM) หรือเครื่องสถานะจำกัด เป็นแบบจำลอง และการ การคำนวณที่ใช้กำหนดพฤติกรรมของระบบผ่านชุดของสถานะ (States) เปลี่ยนผ่านสถานะ (Transitions) ตามเงื่อนไขที่กำหนด โดยในช่วงเวลาใดเวลาหนึ่ง ระบบจะอยู่ได้เพียงสถานะเดียวเท่านั้น FSM เป็นที่นิยมอย่างกว้างขวางในการพัฒนาเกม เนื่องจากเข้าใจง่าย ใช้ทรัพยากรประมวลผลต่ำ และสามารถปรับแก้พฤติกรรมได้โดยไม่กระทบโครงสร้างทั้งหมด

องค์ประกอบหลักของ FSM ได้แก่ สถานะเริ่มต้น (Initial State) สถานะการ ทำงาน (Operational States) เหตุการณ์หรือเงื่อนไข (Event/Condition) การเปลี่ยนสถานะ (Transition) และการกระทำภายในสถานะ (State Action) ใน โครงงานนี้ FSM ถูกนำมาใช้ควบคุมทั้งตัวละครผู้เล่นและ Al ของศัตรู สำหรับตัวละครผู้เล่นใน Hide-and-Seek Horror Game สถานะที่ใช้ใน FSM ได้แก่ การยืนเดิน (Walk) ความเร็วปกติและรัศมีเสียงกว้าง การนั่งย่อ(Crouch) ความเร็วลดลงและรัศมีเสียงปานกลาง และการหมอบคลาน (Prone) ความเร็วต่ำที่สุดและรัศมีเสียงเงียบที่สุด แต่ละสถานะส่งผลต่อขนาด Collider และระดับ Noise Radius ซึ่งเป็นปัจจัยหลักในการถูกตรวจจับโดย A สำหรับ Al ของศัตรู (Ghost/Killer) สถานะที่ใช้ใน FSM ได้แก่ Patrol State (ลาดตระเวนตาม Waypoints), Investigate State (ตรวจสอบจุดที่ ได้ยินเสียง), Chase State (ไล่ล่าเมื่อเห็นผู้เล่นใน Vision Cone) และ Attack State (โจมตีเมื่อระยะห่าง ≤ 2 pixel) นอกจากนี้ FSM ยังกล่าวถึง Hierarchical Finite State Machine (HFSM) ซึ่งเป็น

การจัดลำดับชั้นของสถานะเพื่อลดความ ซับซ้อนของ FSM แบบดั้งเดิม ทำให้สามารถบริหารจัดการพฤติกรรมที่ซับซ้อนได้ดียิ่งขึ้น

2.6 การพัฒนาปัญญาประดิษฐ์ในเกม

Game Al (Game Artificial Intelligence) หมายถึง ระบบที่ควบคุม พฤติกรรมของตัวละครในเกมให้สามารถโต้ตอบกับผู้เล่นได้อย่างสมเหตุสมผล เป้าหมาย หลักของ Game Al ไม่ใช่การเรียนรู้ชั้นสูงแบบ Machine Learning แต่เป็นการ สร้าง "ภาพลวงตาของความฉลาด" เพื่อเพิ่มความสนุกและความท้าทายในการเล่น เทคนิคที่นิยมใช้ใน Game Al ได้แก่ Rule-based System, Finite State Machine (FSM), Behavior Tree และ Pathfinding Algorithm ในเกม 2 มิติแนว Hide-and-Seek Horror Game ระบบ Al ของศัตรูจำเป็นต้อง ผสมผสานเทคนิคหลายอย่างเข้าด้วยกัน ดังนี้

2.6.1 Finite State Machine (FSM) สำหรับศัตรู ใช้ FSM ในการจัดการสถานะพฤติกรรมของศัตรู (Ghost/Killer Al) โดยแบ่ง ออกเป็นสถานะ Patrol, Investigate, Chase และ Attack โดยแต่ละสถานะจะเปลี่ยนไปตามเงื่อนไขที่ตรวจจับได้จากสภาพแวดล้อม เช่น ระยะสายตา และระดับเสียงของผู้เล่น

2.6.2 Sensory System (ระบบการรับรู้) พัฒนาระบบรับรู้ของ Al โดยใช้ Raycasting ในการจำลองสายตา (Line of Sight / Vision Cone) เพื่อตรวจสอบว่าผู้เล่นอยู่ในแนวสายตาของศัตรูหรือไม่ และใช้การคำนวณระยะทาง (Distance Check) เปรียบเทียบกับ Noise Radius ที่ผู้เล่นสร้างขึ้นตามท่าทางการเคลื่อนไหว

2.6.3 การค้นหาเส้นทาง ใช้ระบบ Navigation ร่วมกับ A* Algorithm ให้ Al สามารถเดินไล่ล่าผู้เล่น หลบสิ่งกีดขวาง และเคลื่อนที่ไปยังจุดที่ได้ยินเสียงล่าสุดได้อย่างสมจริงภายในฉาก เกม 2 มิติแนว Side-scrolling

2.7 ภาษาและเครื่องมือที่ใช้พัฒนา

2.7.1 Visual Studio Code

เป็นโปรแกรมแก้ไขโค้ด (Code Editor) ที่พัฒนาโดย Microsoft มีความ ยืดหยุ่นสูง รองรับภาษาโปรแกรมต่างๆ ด้วยส่วนขยาย (Extensions) ที่หลาก หลาย สามารถเชื่อมต่อกับภาษา C# และ Unity Game Engine ได้อย่าง สะดวก มีระบบ IntelliSense ช่วยแนะนำโค้ดและแก้ไขข้อผิดพลาดได้อย่างมี ประสิทธิภาพ

2.7.2 เอ็นจิ้นเกม Unity

เป็นแอปพลิเคชันสำหรับการพัฒนาเกม มีความยืดหยุ่นสูง รองรับการพัฒนาเกมทั้ง 2 มิติ และ 3 มิติ โดยสามารถส่งออกเกมไปยัง Windows, Android และ iOS ได้อย่างสะดวก Unity มีเครื่องมือสำคัญดังนี้

2.7.2.1 Scene Editor ใช้ในการออกแบบฉากและจัดวางวัตถุใน เกม รวมถึงการสร้าง Tilemap สำหรับเกม 2 มิติแนว Side-scrolling

2.7.2.2 Animator ใช้ควบคุมและแสดงผลแอนิเมชันของตัวละครใน สถานะต่าง ๆ เช่น Walk, Crouch, Prone

2.7.2.3 Physics Engine 2D จัดการระบบฟิสิกส์ การชน (Collision) และการเคลื่อนที่ของวัตถุในเกม 2 มิติ

2.7.3 ภาษาโปรแกรม C#

เป็นภาษาหลักที่ใช้ในการพัฒนาเกมใน Unity มีโครงสร้างเชิงวัตถุ (OOP) ที่ ชัดเจน รองรับการสร้างระบบที่ซับซ้อน เช่น ระบบ FSM ระบบ AI ระบบ Inventory และการจัดการสถานะตัวละคร โดยในโครงงานนี้ใช้ C# เขียน สคริปต์ควบคุมการทำงานทั้งหมด

2.7.3.1 Game Logic Scripting สำหรับเขียนตรรกะการทำงาน ของระบบต่าง ๆ เช่น การเปลี่ยนสถานะ FSM และการตรวจจับ

2.7.3.2 Event Handling ใช้จัดการเหตุการณ์ต่าง ๆ เช่น Input ของผู้เล่น การชน และการเปลี่ยนฉาก

2.7.3.3 Data Management ใช้จัดการข้อมูลไอเทม สถานะผู้เล่น และการส่งต่อข้อมูลข้ามฉาก (Data Persistence)

2.7.4 JSON (JavaScript Object Notation)

เป็นรูปแบบไฟล์ข้อมูลที่มีโครงสร้างแบบคู่ระหว่างชื่อ (Key) และค่า (Value) ซึ่งอ่านและแก้ไขได้ง่ายในรูปแบบ Human-Readable Format สามารถใช้ ร่วมกับภาษา C# ใน Unity เพื่อบันทึกและโหลดข้อมูลเกม เช่น ข้อมูลตัวละคร และความคืบหน้าในการเล่น

2.8 อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm)

ระบบค้นหาเส้นทาง (Pathfinding) มีบทบาทสำคัญในการทำให้ Al ของศัตรู สามารถเคลื่อนที่ภายในฉากเกม 2 มิติ Side-scrolling ได้อย่างมีประสิทธิภาพและ หลบหลีกสิ่งกีดขวางต่าง ๆ ได้อย่างสมจริง A* Algorithm (A-Star) เป็นอัลกอริทึมที่นิยมใช้ในเกม เนื่องจากสามารถ คำนวณเส้นทางที่เหมาะสมที่สุด โดยพิจารณาทั้งค่าใช้จ่ายจริงจากจุดเริ่มต้น (g-cost) และค่าประมาณระยะทางไปยังเป้าหมาย (h-cost) โดย f(n) = g(n) + h(n) ทำให้ ได้เส้นทางที่สั้นที่สุดในเวลาอันรวดเร็ว การทำงานร่วมกันระหว่าง FSM และ A* ใน Hide-and-Seek Horror Game มีลักษณะดังนี้ เมื่อ FSM ของศัตรูเปลี่ยนเป็นสถานะ Investigate ระบบจะ เรียกใช้ A* เพื่อนำทางไปยังตำแหน่งที่ได้ยินเสียงล่าสุด และเมื่อเปลี่ยนเป็นสถานะ Chase ระบบจะคำนวณเส้นทางไปยังตำแหน่งปัจจุบันของผู้เล่นแบบเรียลไทม์ โครงสร้างนี้ช่วยให้ศัตรูมีพฤติกรรมการไล่ล่าที่สมจริงและน่าตื่นเต้นยิ่งขึ้น

2.9 การออกแบบ UX/UI ให้เหมาะสม

การออกแบบส่วนติดต่อผู้ใช้ (User Interface: UI) และประสบการณ์ผู้ใช้ (User Experience: UX) เป็นองค์ประกอบสำคัญในการพัฒนาเกมที่ช่วยให้ผู้เล่น สามารถโต้ตอบกับระบบได้อย่างสะดวกและเข้าใจง่าย ในเกม 2 มิติ Side-scrolling Hide-and-Seek Horror Game การออกแบบ UI ที่ดีจะต้องไม่บดบังทัศนวิสัย ของผู้เล่น แต่ยังคงสื่อสารข้อมูลสำคัญได้อย่างชัดเจน องค์ประกอบ UI ที่สำคัญในเกมประเภทนี้ ได้แก่ ตัวบ่งชี้ท่าทาง (Stance Indicator) ที่แสดงสถานะปัจจุบันของผู้เล่นว่ากำลังยืน นั่งย่อ หรือหมอบคลาน มิเตอร์ เสียง (Noise Meter) ที่แสดงระดับความดังของเสียงที่ผู้เล่นสร้างขึ้นแบบเรียลไทม์ พลังชีวิต (HP Bar) และตัวนับไอเทม (Item Counter) เพื่อให้ผู้เล่นทราบความคืบ หน้าของภารกิจ หลักการออกแบบ UX ที่ดีในเกม Horror คือการใช้สีและเสียงเพื่อสร้าง บรรยากาศความกลัว โดยไม่ทำให้ UI ดูไม่สอดคล้องกับโทนของเกม นอกจากนี้ควร ออกแบบ HUD (Head-Up Display) ที่กระชับ สื่อความหมายได้ชัดเจน และตอบ สนองต่อสถานการณ์ในเกมได้อย่างทันท่วงที และน่าตื่นเต้น เพื่อสร้างประสบการณ์การเล่นที่น่าจดจำและน่าตื่นเต้น

2.10 งานวิจัยที่เกี่ยวข้อง

2.10.1 Finite State Machine in Game Development

งานวิจัยนี้ศึกษาแนวคิดและหลักการทำงานของ Finite State Machine (FSM) ในการพัฒนาระบบปัญญาประดิษฐ์ (AI) สำหรับควบคุมพฤติกรรมของตัวละครที่ไม่ใช่ผู้เล่น (Non-Player Characters: NPC) ภายในเกม โดยอธิบายว่า FSM เป็นแบบจำลองที่ประกอบด้วย “สถานะ (State)” และ “การเปลี่ยนสถานะ (Transition)” ซึ่งจะเปลี่ยนไปตามเงื่อนไขที่กำหนดไว้ล่วงหน้า การประยุกต์ใช้ FSM ในเกมช่วยให้สามารถกำหนดพฤติกรรมของตัวละคร เช่น การเดิน การไล่ล่า การโจมตี การป้องกัน หรือการหลบหนี ได้อย่างเป็นระบบ ลดความซับซ้อนของโค้ด และเพิ่มความชัดเจนในการออกแบบระบบ นอกจากนี้ งานวิจัยยังกล่าวถึง Hierarchical Finite State Machine (HFSM) ซึ่งเป็นการจัดลำดับชั้นของสถานะเพื่อลดความซับซ้อนของ FSM แบบดั้งเดิม ทำให้สามารถบริหารจัดการพฤติกรรมที่ซับซ้อนได้ดียิ่งขึ้น ผลการศึกษาสรุปว่า FSM เป็นเทคนิคที่มีประสิทธิภาพ เข้าใจง่าย และยังคงได้รับความนิยมในการพัฒนาเกมสมัยใหม่

2.10.2 Development of Two Dimension (2D) Game Engine with Finite State Machine (FSM) Based Artificial Intelligence (AI) Subsystem

งานวิจัยนี้มุ่งพัฒนาเกมเอนจินแบบ 2 มิติ ที่มีระบบ AI ภายในโดยใช้แนวคิด FSM เป็นแกนหลักของการควบคุมพฤติกรรมตัวละคร พัฒนาด้วยเทคโนโลยีเว็บ เช่น JavaScript และ WebGL 2.0 มีการเปรียบเทียบประสิทธิภาพระหว่างการพัฒนา AI แบบทั่วไป (Naive Approach) กับการใช้ FSM เป็นโครงสร้างหลัก พบว่า การใช้ FSM สามารถรักษาค่าเฟรมเรต (Frames Per Second: FPS) ได้ที่

ประมาณ 60 FPS ขณะที่วิธีทั่วไปทำให้เฟรมเรต ลดลงเหลือประมาณ 50 FPS แสดงให้เห็นว่า FSM ช่วยเพิ่มประสิทธิภาพของระบบ ลดความกำกวมของตรรกะการทำงาน และช่วยให้การจัดการพฤติกรรมของตัวละครเป็นระบบมากขึ้น

2.10.3 An Analysis of Finite State Machine Based Enemy Artificial Intelligence in Kirby: Nightmare in Dream Land

บทความนี้ศึกษาการประยุกต์ใช้เทคนิค **Finite State Machine (FSM)** ในการออกแบบปัญญาประดิษฐ์ของศัตรูในเกม Kirby: Nightmare in Dream Land โดยมุ่งวิเคราะห์ว่าโครงสร้างแบบกำหนดสถานะสามารถควบคุมพฤติกรรมศัตรูและส่งผลต่อระดับความยากของเกมได้อย่างไร ผู้วิจัยอธิบายว่า FSM เป็นโมเดลที่แบ่งพฤติกรรมออกเป็นสถานะต่าง ๆ เช่น หยุดนิ่ง (Idle) เดินลาดตระเวน (Patrol) โจมตี (Attack) ถูกโจมตี (Hit) และฟื้นตัว (Recover) และกำหนดเงื่อนไขการเปลี่ยนสถานะอย่างชัดเจนตามเหตุการณ์ที่เกิดขึ้นในเกม จากการทดลองสังเกตพฤติกรรมศัตรูหลายครั้ง พบว่าสถานะการโจมตีมีสัดส่วนเวลามากที่สุด ขณะที่สถานะหยุดนิ่งและเดินช่วยสร้างจังหวะพักให้ผู้เล่น ทำให้เกมมีความสมดุล ไม่ยากหรือไม่ง่ายเกินไป นอกจากนี้ ผู้วิจัยยังทดลองปรับค่าระยะเวลาหน่วงก่อนการโจมตี (Attack Cooldown) และพบว่ายิ่งศัตรูโจมตีถี่ขึ้น ระดับความยากของเกมก็เพิ่มสูงขึ้นอย่างชัดเจน แสดงให้เห็นว่า FSM สามารถปรับความท้าทายของเกมได้โดยการแก้ไขพารามิเตอร์เพียงบางค่า โดยไม่ต้องปรับโครงสร้าง AI ทั้งระบบ โดยสรุป งานวิจัยชี้ให้เห็นว่า FSM เป็นเทคนิคที่มีความเสถียร ใช้ทรัพยากรประมวลผลต่ำ ควบคุมและปรับแต่งได้ง่าย และยังคงมีความเหมาะสมสำหรับการพัฒนา AI ในเกมแพลตฟอร์ม 2 มิติ แม้จะมีเทคนิค AI ที่ซับซ้อนกว่าพัฒนาขึ้นในปัจจุบันก็ตาม

2.10.4 การออกแบบเกมดิจิทัล

งานวิจัยนี้นำเสนอแนวคิดและกระบวนการออกแบบเกมดิจิทัล โดยกล่าวถึงหลักการออกแบบจากหลากหลายแนวทาง เช่น การออกแบบจากนวัตกรรม ความคิดสร้างสรรค์ เรื่องราว ความต้องการของผู้เล่น และข้อจำกัดด้านเทคโนโลยี นอกจากนี้ยังกล่าวถึงประเภทของเกม เช่น เกม 2 มิติ และ 3 มิติ รวมถึงการแบ่งตามลักษณะการเล่น เช่น เกมต่อสู้ เกมยิง เกมผจญภัย และเกมการศึกษา พร้อมทั้งเสนอขั้นตอนการพัฒนาเกมอย่างเป็นระบบ ได้แก่ การวางแผน การจัดทำเอกสารออกแบบเกม (Game Design Document: GDD) การพัฒนาโปรแกรม การสร้างวัตถุในเกม การทดสอบ และการเผยแพร่ ซึ่งเป็นแนวทางสำคัญในการพัฒนาเกมให้ประสบความสำเร็จ

2.10.5 การพัฒนาเกมเพื่อการเรียนรู้การเขียนโปรแกรมภาษา C# ด้วยเกมการผจญภัยของอาเรียกับเวทย์มนต์แห่ง C#

งานวิจัยนี้มีวัตถุประสงค์เพื่อพัฒนาเกมสำหรับเสริมการเรียนรู้การเขียนโปรแกรมภาษา C# โดยใช้โปรแกรม Unity ในการพัฒนาเกมแนวผจญภัย (Adventure Game) และใช้แนวคิด Game-Based Learning เพื่อให้ผู้เรียนเรียนรู้ผ่านการเล่นเกม ผลการประเมินโดยผู้เชี่ยวชาญจำนวน 5 คน

พบว่าคุณภาพของเกมอยู่ในระดับดีมาก (ค่าเฉลี่ย 4.71) และเมื่อนำไปทดลองกับกลุ่มตัวอย่างจำนวน 20 คน พบว่าคะแนนหลังเรียนเพิ่มขึ้นร้อยละ 40.92 แสดงให้เห็นว่าเกมสามารถช่วยเพิ่มความเข้าใจในการเขียนโปรแกรม และส่งเสริมการคิดอย่างเป็นระบบได้อย่างมีประสิทธิภาพ

2.10.6 A Systematic Review and Analysis of Intelligence-Based Pathfinding Algorithms in the Field of Video Games

งานวิจัยนี้นำเสนอการทบทวนและวิเคราะห์อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm) ที่ใช้ปัญญาประดิษฐ์ในเกมวิดีโอ โดยจำแนกอัลกอริทึมออกเป็น 3 กลุ่มหลัก ได้แก่ กลุ่ม Uninformed Search เช่น BFS และ Dijkstra's กลุ่ม Informed Search เช่น A* และ Greedy Best First และกลุ่ม Metaheuristic เช่น Genetic Algorithm และ Ant Colony Optimization ผู้วิจัยได้ทดสอบอัลกอริทึมบน Grid 2 มิติขนาด 4×4, 8×8, 16×16 และ 32×32 โดยใช้ Unity Game Engine เป็นสภาพแวดล้อมในการทดสอบ วัดผลด้วยเวลาประมวลผล (Execution Time) จำนวนบล็อกที่ผ่าน และความยาวเส้นทางที่สั้นที่สุด ผลการทดลองพบว่า A* Algorithm มีความสมดุลระหว่างความเร็วและความแม่นยำสูงที่สุด โดยใช้เวลาประมวลผลน้อยกว่า Dijkstra's และ BFS ในทุกขนาด Grid ทั้งที่มีและไม่มีสิ่งกีดขวาง และยังรับประกันเส้นทางที่สั้นที่สุด ซึ่งต่างจาก Greedy Best First ที่แม้จะเร็วที่สุดแต่ไม่ให้เส้นทางที่เหมาะสมที่สุดเสมอไป งานวิจัยนี้ยืนยันว่า A* เป็นอัลกอริทึมที่เหมาะสมที่สุดสำหรับการนำทาง NPC ในเกม 2 มิติ ซึ่งสอดคล้องกับการออกแบบระบบ AI ของศัตรูในโครงงานนี้

2.10.7 Improving Mobile Game Performance with Basic Optimization Techniques in Unity

งานวิจัยนี้ศึกษาผลกระทบของเทคนิค Optimization พื้นฐานที่มีต่อประสิทธิภาพของเกมบนอุปกรณ์มือถือ Android โดยพัฒนาเกม Shooter 3 มิติด้วย Unity และ Blender จำนวน 2 เวอร์ชัน ได้แก่ เวอร์ชันที่ใช้เทคนิค Optimization และเวอร์ชันที่ไม่ใช้ โดยเทคนิคที่นำมาทดสอบประกอบด้วย Low Poly Model, Merge Vertices, Textures and Materials, Occlusion Culling และ Object Pooling ตัวชี้วัดประสิทธิภาพที่ใช้คือค่า Frames Per Second (FPS), Batches และจำนวน Triangles/Polygons ซึ่งสะท้อนการใช้งาน CPU, GPU และหน่วยความจำตามลำดับ ผลการทดสอบบนอุปกรณ์มือถือจริงพบว่าเวอร์ชันที่ใช้เทคนิค Optimization รักษาค่า FPS ได้ในช่วง 50–60 FPS ตลอดการเล่น ขณะที่เวอร์ชันที่ไม่ผ่านการ Optimize มีค่า FPS ตกลงเหลือเพียง 20–30 FPS เมื่อมีศัตรูจำนวนมากบนหน้าจอ ทำให้เกมเกิดอาการกระตุก งานวิจัยนี้ยืนยันว่าการเลือกใช้เทคนิค Optimization ที่เหมาะสมมีผลโดยตรงต่อความลื่นไหลของเกม ซึ่งสอดคล้องกับโครงงานนี้ที่ต้องการให้เกม Side-scrolling Horror ทำงานได้อย่างมีประสิทธิภาพบน Unity

บทที่ 3

วิธีการดำเนินงาน

เทคนิค Finite State Machine ในการควบคุมตัวละครและ ปัญญาประดิษฐ์ ด้วยภาษา C# มุมมอง Side-scrolling คณะวิทยาศาตสร์และเทคโนโลยีผู้พัฒนาได้ทำการศึกษาถึงหลักการ และทฤษฎีต่าง ๆ ในการพัฒนาระบบงาน และดำเนินการตามขั้นตอนของวงจรการพัฒนาระบบและดำเนินการ ตามหลักการออกแบบ (System Development Life Cycle : SDLC) มี 7 ขั้นตอน ดังนี้

3.1 กำหนดปัญหา
3.2 การรวบรวมข้อมูล
3.3 การวิเคราะห์ระบบ
3.4 การออกแบบระบบ
3.5 การพัฒนาระบบ
3.6 การทดสอบ
3.7 การบำรุงรักษา

3.1 กำหนดปัญหา

จากแนวคิดและวัตถุประสงค์ของโครงงานในบทที่ 1 ซึ่งมุ่งเน้นการพัฒนาเกมคอมพิวเตอร์ 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game ด้วยภาษา C# พบว่ายังมีประเด็นปัญหาที่ต้องพิจารณาในการพัฒนาระบบดังต่อไปนี้

3.1.1 ปัญหาด้านโครงสร้างการควบคุมตัวละครผู้เล่น การควบคุมตัวละครผู้เล่นในเกมมีหลายสถานะ เช่น ยืนนิ่ง เดิน ย่อตัว และนอนราบ ซึ่งแต่ละสถานะมีพฤติกรรมและเงื่อนไขการเปลี่ยนสถานะที่แตกต่างกัน หากไม่มีการออกแบบโครงสร้างที่เหมาะสม อาจทำให้การเขียนโปรแกรมมีความซับซ้อนและเกิดการซ้ำซ้อนของโค้ด นอกจากนี้การเปลี่ยนสถานะที่ไม่เป็นระบบอาจส่งผลให้ตัวละครเกิดพฤติกรรมที่ไม่ถูกต้อง เช่น เปลี่ยนสถานะผิดเงื่อนไขหรือเกิดการค้างของสถานะ รวมถึงระบบยังต้องรองรับการตอบสนองแบบเรียลไทม์ที่ต้องการความแม่นยำและรวดเร็ว

3.1.2 ปัญหาด้านการออกแบบพฤติกรรมของศัตรู (Enemy AI) ระบบปัญญาประดิษฐ์ของศัตรูต้องสามารถตอบสนองต่อการกระทำของผู้เล่นได้อย่างสมจริง เช่น การตรวจจับ การไล่ล่า และการโจมตี หากการออกแบบพฤติกรรมไม่มีโครงสร้างที่ชัดเจน อาจทำให้ศัตรูมีพฤติกรรมไม่สมเหตุสมผลหรือทำงานผิดลำดับ การเปลี่ยนสถานะของศัตรูระหว่างการลาดตระเวน การตรวจสอบ การไล่ล่า และการโจมตีจำเป็นต้องมีเงื่อนไขที่ถูกต้องและสอดคล้องกับสถานการณ์ และการประมวลผลต้องคำนึงถึงประสิทธิภาพเพื่อไม่ให้ส่งผลกระทบต่อการทำงานของเกมโดยรวม

3.1.3 ปัญหาด้านระบบการตรวจจับ (Detection System) การตรวจจับผู้เล่นของศัตรูต้องพิจารณาจากหลายปัจจัย เช่น ระยะทาง มุมมอง และระดับเสียง หากไม่มีการกำหนดเงื่อนไขที่เหมาะสม อาจทำให้ศัตรูตรวจจับผู้เล่นได้ง่ายหรือยากเกินไป ส่งผลต่อความสมดุลของเกม การจำลองการมองเห็นด้วย Raycasting ต้องมีความแม่นยำในการตรวจสอบสิ่งกีดขวาง และระบบการได้ยินต้องสัมพันธ์กับพฤติกรรมของผู้เล่น เช่น การเดินจะมีเสียงมากกว่าการหมอบ

3.1.4 ปัญหาด้านการเคลื่อนที่และการค้นหาเส้นทางของศัตรู ศัตรูต้องสามารถเคลื่อนที่ภายในฉากที่มีสิ่งกีดขวางได้อย่างเหมาะสม หากไม่มีระบบค้นหาเส้นทางที่มีประสิทธิภาพ อาจทำให้ศัตรูติดสิ่งกีดขวางหรือเคลื่อนที่ผิดพลาด การไล่ล่าผู้เล่นแบบเรียลไทม์จำเป็นต้องมีการคำนวณเส้นทางอย่างต่อเนื่องเพื่อให้ศัตรูสามารถตอบสนองต่อสถานการณ์ที่เปลี่ยนแปลงได้อย่างทันท่วงที

3.1.5 ปัญหาด้านการออกแบบระบบเกม (Game Logic) การกำหนดเงื่อนไขของเกม เช่น การเก็บไอเทมและการปลดล็อกด่าน ต้องมีความชัดเจนและไม่ซับซ้อนเกินไป หากไม่มีการจัดการข้อมูลที่ดีอาจทำให้เกิดข้อผิดพลาดในการตรวจสอบเงื่อนไข การเปลี่ยนฉากต้องสามารถส่งต่อข้อมูลของผู้เล่นได้อย่างถูกต้อง และการออกแบบลำดับเหตุการณ์ของเกมต้องทำให้ผู้เล่นเข้าใจง่ายและเล่นได้อย่างต่อเนื่อง

3.1.6 ปัญหาด้านส่วนติดต่อผู้ใช้งาน (UI/UX) ผู้เล่นจำเป็นต้องรับรู้ข้อมูลสำคัญ เช่น สถานะตัวละคร ระดับเสียง และจำนวนไอเทม แบบเรียลไทม์ หากการแสดงผลไม่ชัดเจนหรือรบกวนการเล่น อาจทำให้ผู้เล่นไม่สามารถวางแผนการเล่นได้อย่างมีประสิทธิภาพ การออกแบบจึงต้องคำนึงถึงความเข้าใจง่าย ความสะดวกในการใช้งาน และความสัมพันธ์กับกลไกของเกม เช่น ระดับเสียงที่ส่งผลต่อการถูกตรวจจับ

3.1.7 ปัญหาด้านประสิทธิภาพของระบบ (Performance) การประมวลผลหลายระบบพร้อมกัน เช่น การเคลื่อนที่ การตรวจจับ และการตัดสินใจของศัตรู อาจทำให้เกิดความล่าช้าและส่งผลต่ออัตราเฟรมเรตของเกม ดังนั้นการออกแบบระบบที่มีความซับซ้อนต่ำแต่มีประสิทธิภาพสูงจึงเป็นสิ่งสำคัญ โดยต้องมีการจัดการหน่วยความจำและการประมวลผลอย่างเหมาะสม เพื่อให้ระบบสามารถทำงานได้อย่างต่อเนื่องโดยไม่เกิดข้อผิดพลาด

3.1.8 ปัญหาด้านการขยายระบบในอนาคต (Scalability) หากโครงสร้างของระบบไม่ยืดหยุ่นเพียงพอ การเพิ่มฟังก์ชันใหม่ เช่น การเพิ่มสถานะของตัวละครหรือพฤติกรรมของศัตรู อาจทำได้ยากและใช้เวลามาก การออกแบบแบบ Modular จึงเป็นสิ่งจำเป็นเพื่อแยกส่วนการทำงานออกจากกัน และต้องสามารถปรับแต่งค่าพารามิเตอร์ได้ง่าย เช่น ความเร็วหรือระยะตรวจจับ เพื่อให้ระบบสามารถนำไปพัฒนาต่อยอดได้ในอนาคต

3.2 การรวบรวมข้อมูล

การศึกษารวบรวมข้อมูลที่มีความเกี่ยวข้องกับการพัฒนาเกมคอมพิวเตอร์ 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game ด้วย C# ผู้จัดทำได้ทำการศึกษาค้นคว้าข้อมูลที่เกี่ยวข้องจากแหล่งข้อมูลต่าง ๆ เช่น งานวิจัย อินเทอร์เน็ต หนังสือ เป็นต้น โดยได้รวบรวมข้อมูลที่เกี่ยวข้อง และสรุปข้อมูลที่จะนำไปใช้ในการจัดทำครั้งนี้ ทางผู้พัฒนาได้ทำการรวบรวมข้อมูลมีดังต่อไปนี้

3.2.1 รวบรวมข้อมูลที่เกี่ยวกับทฤษฎีที่เกี่ยวข้องกับการพัฒนาเกมคอมพิวเตอร์ 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game รวมถึงหลักการออกแบบและการนำเทคนิค Finite State Machine (FSM) มาประยุกต์ใช้ในการควบคุมสถานะตัวละครผู้เล่น ได้แก่ สถานะ Idle Walk Crouch และ Prone และการนำ FSM มาใช้เป็นโครงสร้างหลักในการจัดการพฤติกรรมของ AI ศัตรู ได้แก่ สถานะ Patrol Investigate Chase และ Attack เพื่อให้สามารถพัฒนาระบบการทำงานของเกมได้อย่างมีประสิทธิภาพ

3.2.2 ทำการศึกษาอุปกรณ์และโปรแกรมที่จะใช้ในการพัฒนาการทำงานของระบบ ทางผู้วิจัยได้ทำการศึกษาและเลือกใช้อุปกรณ์และโปรแกรมต่าง ๆ ที่จำเป็นในการพัฒนาเกม ดังนี้ ใช้ Unity Game Engine ในการพัฒนาเกม 2 มิติ ซึ่งรองรับระบบ Sprite, Tilemap, Physics 2D, Animator และ Navigation System ได้อย่างครบถ้วน ใช้ภาษา C# ในการเขียน Script ควบคุมการทำงานทั้งหมด ครอบคลุมระบบ FSM ระบบ AI ระบบ Inventory และการจัดการสถานะตัวละคร ใช้ JSON ในการเก็บและโหลดข้อมูลเกม เช่น ข้อมูลตัวละครและความคืบหน้าในการเล่น นอกจากนี้ยังใช้หลักการ OOP (Object-Oriented Programming) และ Design Patterns เพื่อเพิ่มความยืดหยุ่นและสามารถขยายระบบได้ในอนาคต รวมถึงการใช้ Unit Testing และ Debugging Tool ใน Unity Game Engine เพื่อตรวจสอบข้อผิดพลาดในโค้ดและการทำงานของระบบต่าง ๆ

3.3 การวิเคราะห์ระบบ

จากการศึกษาขั้นตอนการพัฒนาเกมคอมพิวเตอร์ 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game ด้วยภาษา C# ผู้พัฒนาได้ทำการวิเคราะห์โครงสร้างของระบบและการทำงานร่วมกันของแต่ละส่วน เพื่อให้ระบบทั้งหมดทำงานได้อย่างราบรื่นและมีประสิทธิภาพ โดยสามารถแบ่งการวิเคราะห์ออกเป็นส่วนหลักดังนี้

3.3.1 การวิเคราะห์ระบบควบคุมตัวละครผู้เล่น (Player Controller System) ระบบควบคุมตัวละครผู้เล่นต้องรองรับการเคลื่อนไหวหลายรูปแบบ ได้แก่ Idle (ยืนนิ่ง) Walk (เดิน) Crouch (นั่งย่อ) และ Prone (หมอบคลาน) โดยแต่ละสถานะส่งผลต่อค่าพารามิเตอร์ภายในเกมแบบเรียลไทม์ ได้แก่ ความเร็วในการเคลื่อนที่ (Movement Speed) ขนาด Collider และค่า Noise Radius ซึ่งส่งผลโดยตรงต่อโอกาส

การถูกตรวจจับจากศัตรู จึงนำเทคนิค Finite State Machine (FSM) มาใช้เป็นโครงสร้างหลักในการจัดการสถานะ เพื่อให้การเปลี่ยนสถานะเป็นไปอย่างเป็นระบบและมีเงื่อนไขที่ชัดเจน

3.3.2 การวิเคราะห์ระบบปัญญาประดิษฐ์ของศัตรู (Enemy AI System) ระบบ AI ของศัตรูต้องสามารถตอบสนองต่อพฤติกรรมของผู้เล่นได้อย่างสมจริง โดยใช้ FSM ในการจัดการสถานะพฤติกรรมของศัตรู ได้แก่ Patrol (ลาดตระเวน) Investigate (ตรวจสอบ) Chase (ไล่ล่า) และ Attack (โจมตี) ซึ่งการเปลี่ยนสถานะแต่ละครั้งขึ้นอยู่กับข้อมูลที่ได้รับจากระบบการรับรู้ของศัตรู ประกอบด้วยระบบการมองเห็นผ่าน Vision Cone โดยใช้เทคนิค Raycasting ในการตรวจสอบว่าผู้เล่นอยู่ในแนวสายตาของศัตรูหรือไม่ และระบบการได้ยินเสียงโดยคำนวณระยะทางเปรียบเทียบกับค่า Noise Radius ที่ผู้เล่นสร้างขึ้นตามสถานะการเคลื่อนไหว เมื่อตรวจจับได้ ระบบ FSM ของศัตรูจะเปลี่ยนสถานะและเรียกใช้ A* Algorithm เพื่อคำนวณเส้นทางในการเคลื่อนที่ไปยังตำแหน่งเป้าหมายได้อย่างสมจริง

3.3.3 การวิเคราะห์ระบบเกม (Game Logic System) ระบบเกมทำหน้าที่ควบคุมกลไกหลักของเกม ประกอบด้วยระบบ Inventory ที่ใช้โครงสร้างข้อมูลแบบ List ในการจัดเก็บไอเทม และตรวจสอบเงื่อนไขการเก็บไอเทมครบ 5 ชิ้นเพื่อปลดล็อกประตูทางออก รวมถึงระบบ Scene Management ที่รองรับการโหลดฉากใหม่และการส่งต่อข้อมูลตัวละครข้ามฉาก (Data Persistence) โดยใช้รูปแบบ JSON ในการจัดเก็บข้อมูล เพื่อให้การดำเนินเกมเป็นไปอย่างต่อเนื่องและสมบูรณ์

3.3.4 การวิเคราะห์ระบบส่วนติดต่อผู้ใช้งาน (UI/UX System) ระบบ UI ต้องแสดงข้อมูลสำคัญแบบเรียลไทม์โดยไม่บดบังพื้นที่เกม ประกอบด้วย HP Bar แสดงพลังชีวิตของผู้เล่น Noise Meter แสดงระดับเสียงที่ผู้เล่นสร้างขึ้น Stance Indicator แสดงสถานะท่าทางปัจจุบัน และตัวนับจำนวนไอเทมที่เก็บได้ การแสดงผลเหล่านี้ช่วยให้ผู้เล่นสามารถวางแผนการเคลื่อนที่และตัดสินใจได้อย่างมีกลยุทธ์

3.3.5 การวิเคราะห์การทำงานร่วมกันของระบบ (System Integration) ระบบทั้งหมดถูกออกแบบให้ทำงานเชื่อมโยงกันอย่างเป็นลำดับ โดยระบบควบคุมตัวละครจะรับข้อมูลจากผู้เล่นและส่งต่อข้อมูล เช่น ตำแหน่งและระดับเสียง ไปยังระบบ AI ของศัตรูเพื่อใช้ในการประมวลผลการตรวจจับและตัดสินใจเปลี่ยนสถานะ ในขณะเดียวกันระบบตรรกะของเกมจะคอยตรวจสอบเงื่อนไขต่าง ๆ และข้อมูลทั้งหมดจะถูกแสดงผลผ่านระบบ UI แบบเรียลไทม์ การเชื่อมโยงของระบบในลักษณะนี้ช่วยให้เกมมีความสมจริงและสามารถตอบสนองต่อการกระทำของผู้เล่นได้อย่างมีประสิทธิภาพ

3.4 การออกแบบระบบ

จากการศึกษาและวิเคราะห์ระบบการทำงาน ทำให้ทราบถึงกระบวนการในการทำงาน และความต้องการของการพัฒนาเกมในแต่ละส่วน จึงได้ขั้นตอนการจัดทำข้อมูลต่าง ๆ และการออกแบบการพัฒนาเกม มีรายละเอียดในส่วนต่าง ๆ ดังนี้

3.4.1 แนวคิดและหลักการทำงาน

การออกแบบการทำงานของการพัฒนาเกม 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game ด้วย C# ต้องเขียนโครงร่างขั้นตอนการทำงานของระบบโดยการเขียนผังการทำงานเพื่อกำหนดแนวทางและขั้นตอนการทำงานที่ต้องการ

3.4.1.1 การนำเข้าข้อมูลเป็นกระบวนการสำคัญในการพัฒนาเกม 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game ด้วย C# โดยข้อมูลที่จำเป็นต้องมีในการพัฒนาเกม ได้แก่ ข้อมูลตัวละคร ข้อมูลศัตรู และข้อมูลอื่น ๆ ที่เกี่ยวข้อง

3.4.1.1.1 ข้อมูลตัวละครประกอบด้วย ชื่อ รายละเอียดตัวละคร ท่าทางที่ใช้ได้ และค่าพารามิเตอร์ในแต่ละสถานะ ได้แก่ ความเร็วในการเคลื่อนที่ (Movement Speed) ขนาด Collider และรัศมีเสียง (Noise Radius) ซึ่งจะถูกนำมาใช้ในการกำหนดคุณสมบัติของตัวละครในเกม

3.4.1.1.2 ข้อมูลศัตรูจะประกอบไปด้วย ประเภทศัตรู ระยะการมองเห็น (Vision Cone) และพารามิเตอร์ต่าง ๆ ของระบบ AI เพื่อใช้กำหนดพฤติกรรมของศัตรูในแต่ละสถานะ ได้แก่ Patrol Investigate Chase และ Attack

3.4.1.1.3 ข้อมูลที่สำคัญต่อการพัฒนาเกมอื่น ๆ เช่น ข้อมูลไอเทม ข้อมูลแผนที่ และข้อมูลเนื้อเรื่อง ซึ่งจำเป็นต้องการสร้างสภาพแวดล้อมของเกม การจัดเก็บข้อมูลเหล่านี้จะช่วยให้การตั้งค่าและการดำเนินเรื่องราวของเกมเป็นไปได้อย่างราบรื่นและสมบูรณ์

3.4.2 ตารางวิเคราะห์สังเคราะห์สำหรับการพัฒนาเกม 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game ด้วย C#

หมวดหมู่	ระบบ	วิเคราะห์	สังเคราะห์
ระบบควบคุมตัวละคร	FSM (Player)	- ตัวละครมีสถานะ Idle, Walk, Crouch และ Prone - แต่ละสถานะส่งผลต่อ Speed, Collider และ Noise Radius	- ทำให้โครงสร้างโค้ดชัดเจน ลดความซับซ้อน - สร้างความท้าทายในการตัดสินใจเลือกท่าทาง
ระบบ AI ศัตรู	FSM (Enemy)	- ศัตรูมีสถานะ Patrol, Investigate, Chase และ Attack - เปลี่ยนสถานะตามการตรวจจับผู้เล่น	- เพิ่มความท้าทายและความสมจริง - พฤติกรรมศัตรูมีเหตุผลและน่าเชื่อถือ

ระบบการมองเห็น	Raycasting	- ใช้ Vision Cone ตรวจสอบแนวสายตา - ขนาด Collider ของผู้เล่นส่งผลต่อการถูกตรวจพบ	- ผู้เล่นสามารถหลบเลี่ยงได้โดยการนั่งย่อหรือหมอบ
ระบบตรวจจับเสียง	Sound Detection	- คำนวณระยะทางเปรียบเทียบกับ Noise Radius - Noise Radius ขึ้นอยู่กับสถานะของผู้เล่น	- สร้างความสนุกในกลยุทธ์การซ่อนตัว - ยิ่งหมอบยิ่งเงียบตรวจจับได้ยากขึ้น
ระบบค้นหาเส้นทาง	A* Pathfinding	- ศัตรูใช้ A* Algorithm นำทางในฉาก - คำนวณเส้นทางใหม่แบบ Real-time ใน Chase State	- ศัตรูเคลื่อนที่หลบสิ่งกีดขวางได้สมจริง
ระบบ Inventory	List Data Structure	- จัดเก็บไอเทมที่เก็บได้ใน List - ตรวจสอบเงื่อนไขครบ 5 ชิ้นเพื่อปลดล็อก	- สร้างเป้าหมายในการเล่นที่ชัดเจน
ระบบบันทึกเกม	JSON Persistence	- บันทึกและโหลดข้อมูลในรูปแบบ JSON - ส่งต่อข้อมูลตัวละครข้ามฉาก	- ผู้เล่นสามารถเล่นต่อจากจุดที่บันทึกได้

3.4.2.1 สังเคราะห์ผล

3.4.2.1.1 ระบบการควบคุมตัวละครด้วย FSM ช่วยให้โครงสร้างโค้ดมีความชัดเจน ลดความซับซ้อน และง่ายต่อการบำรุงรักษาและขยายระบบในอนาคต

3.4.2.1.2 ระบบ AI ของตัวละครศัตรูใช้ FSM ร่วมกับ Sensory System ทำให้ศัตรูตอบสนองต่อสถานการณ์ต่าง ๆ ได้อย่างสมเหตุสมผล

3.4.2.1.3 ระบบค้นหาเส้นทางด้วย A* Algorithm ช่วยให้ศัตรูเคลื่อนที่หลบสิ่งกีดขวางได้อย่างสมจริง โดยใช้สูตร f(n) = g(n) + h(n) ในการคำนวณเส้นทางที่สั้นที่สุด

3.4.2.1.4 ระบบตรวจจับเสียงใช้การคำนวณระยะทาง (Distance Check) เปรียบเทียบกับค่า Noise Radius ที่แตกต่างกันในแต่ละสถานะ เพื่อให้การตรวจจับมีความสมจริง

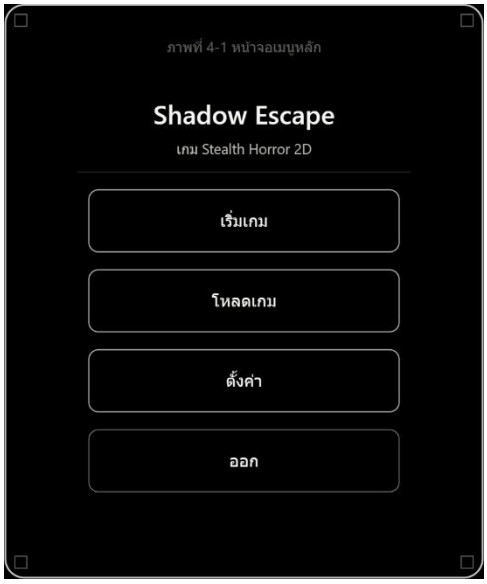
3.4.2.1.5 ระบบเควสต์หลักใช้ List Data Structure ในการตรวจสอบเงื่อนไขการเก็บไอเทม และใช้ Trigger-based Events เพื่อปลดล็อกเป้าหมาย

3.4.2.1.6 ระบบควบคุมใช้ Input Handler ร่วมกับ State Pattern เพื่อทำให้การเปลี่ยนสถานะตัวละครเป็นไปอย่างลื่นไหลและไม่ซับซ้อน

3.4.2.1.7 ระบบ Inventory ใช้ List สำหรับการจัดเก็บไอเทมและตรวจสอบจำนวน

3.4.2.1.8 ระบบเซฟเกมและโหลดเกมใช้การจัดเก็บข้อมูลในรูปแบบ JSON เพื่อให้ผู้เล่นสามารถบันทึกความคืบหน้าและเล่นต่อจากจุดเดิมได้

3.4.3 การออกแบบเอาต์พุต (Output)



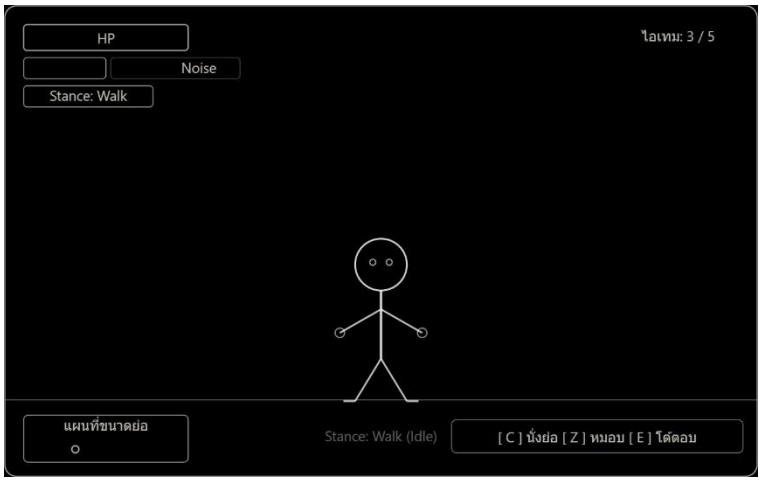
ภาพที่3-1 หน้าจอเมนูหลัก

หน้าจอเมนูหลักของเกม ออกแบบให้มีโทนสีเข้ม เหมาะสมกับบรรยากาศของเกม Stealth Horror โดยแสดงชื่อเกมและคำอธิบายประเภทเกมไว้ด้านบน และมีปุ่มเมนูทั้งหมด 4 ปุ่ม ได้แก่ เริ่มเกม โหลดเกม ตั้งค่า และออก



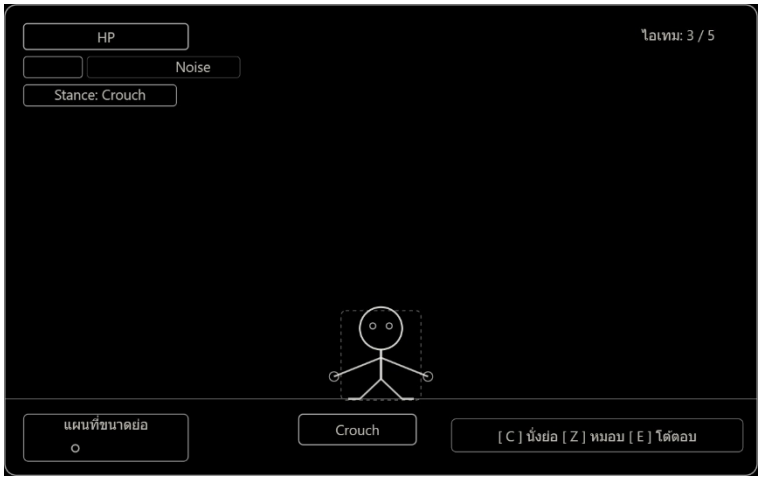
ภาพที่3-2 หน้าจอระหว่างเล่น

หน้าจอระหว่างเล่นออกแบบให้แสดงข้อมูลสำคัญที่มุมต่าง ๆ ของหน้าจอโดยไม่บดบังพื้นที่เกม ประกอบด้วย HP Bar แสดงด้วยสีแดงที่มุมบนซ้าย Noise Meter แสดงด้วยสีส้มบอกระดับเสียงที่ผู้เล่นสร้าง แบบ Real-time Stance Indicator แสดงสถานะท่าทางปัจจุบัน จำนวนไอเทมที่มุมบนขวา และแผนที่ขนาดย่อที่มุมล่างซ้าย พร้อมแสดง Hotkey ที่มุมล่างขวา



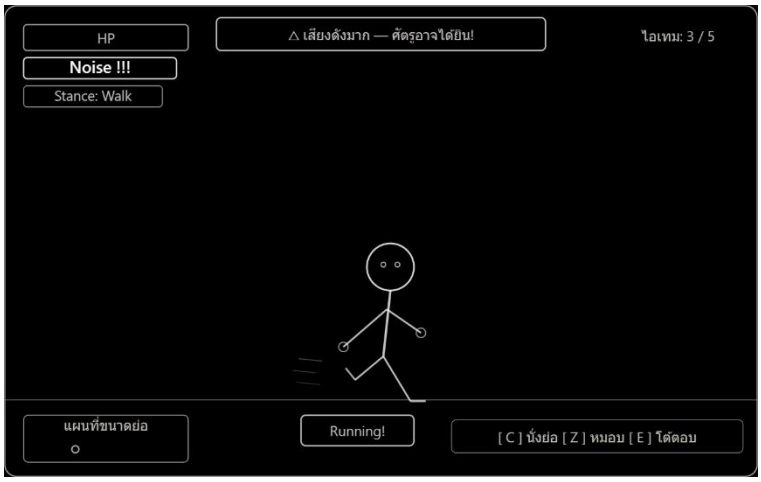
ภาพที่ 3-3 หน้าจอแสดงสถานะตัวละครยืนปกติในสถานะ Walk (Idle)

ซึ่งเป็นสถานะเริ่มต้นของตัวละครผู้เล่น ระบบ FSM จะกำหนดค่าพารามิเตอร์ในสถานะนี้ให้มีความเร็วในการเคลื่อนที่และรัศมีเสียงในระดับปกติ



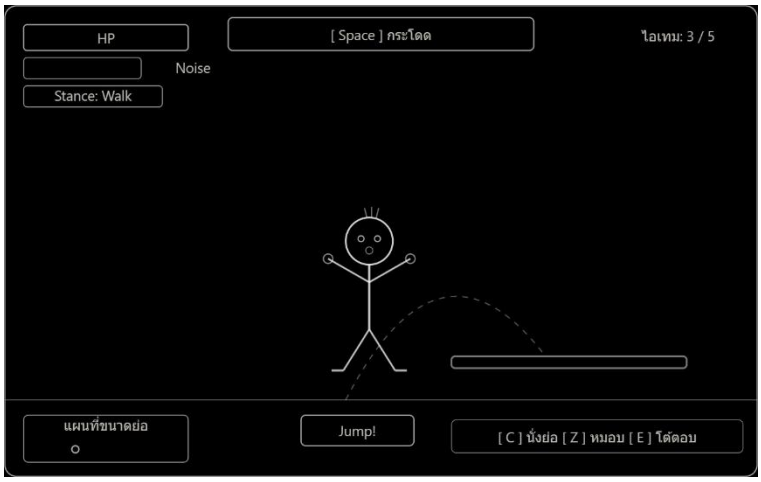
ภาพที่ 3-4 สถานะเมื่อตัวละครย่อตัว (Crouch)

ระบบ FSM จะปรับขนาด Collider ของตัวละครให้เล็กลงแบบ Real-time และลดค่า Noise Radius ลง ส่งผลให้ตัวละครเคลื่อนที่ได้เงียบกว่าสถานะยืนปกติ ทำให้ศัตรูตรวจจับได้ยากขึ้น



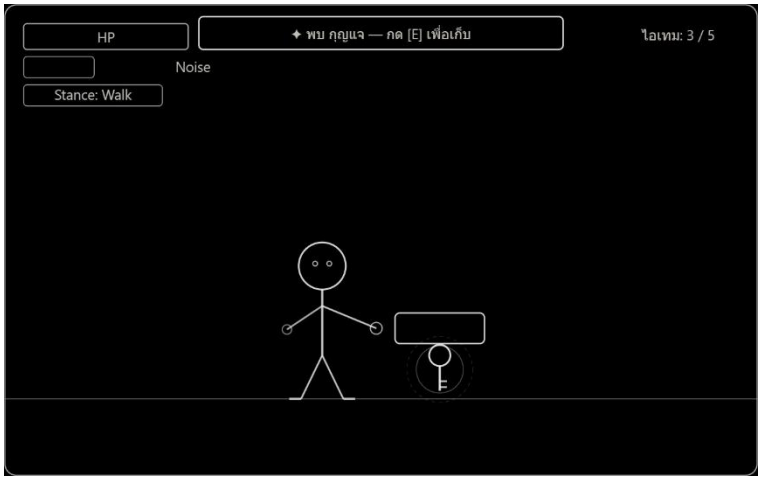
ภาพที่ 3-5 สถานะเมื่อตัวละครวิ่ง

หน้าจอแสดงการเคลื่อนที่ในสถานะ Walk ที่มีการเคลื่อนไหว ระบบจะเพิ่มค่า Noise Radius ให้สูงขึ้นตามความเร็ว ทำให้ศัตรูสามารถตรวจจับเสียงจากระยะไกลได้มากขึ้น ซึ่งสอดคล้องกับการทำงานของระบบ Sound Detection



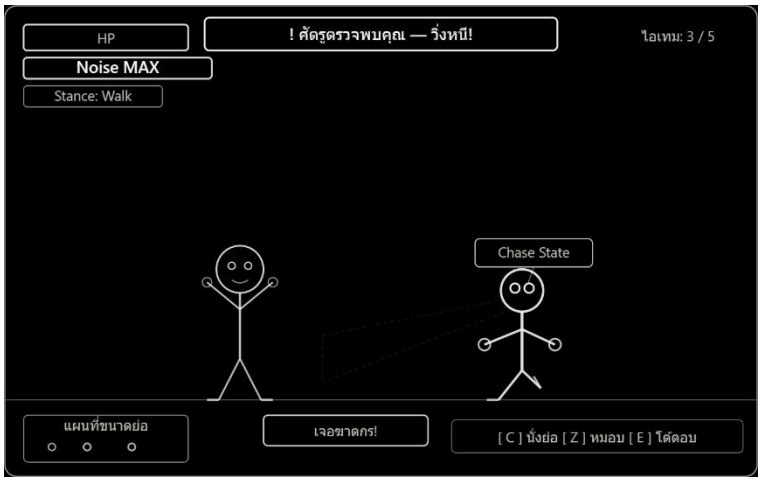
ภาพที่ 3-6 สถานะขณะที่ตัวละครกระโดด

แสดงแอนิเมชันและวิถีการกระโดดของตัวละคร ระบบ Physics 2D ของ Unity จะคำนวณการเคลื่อนที่ตามแรงโน้มถ่วง พร้อมแสดง Hotkey กระตุ้นให้ผู้เล่นทราบปุ่มควบคุมที่เกี่ยวข้อง



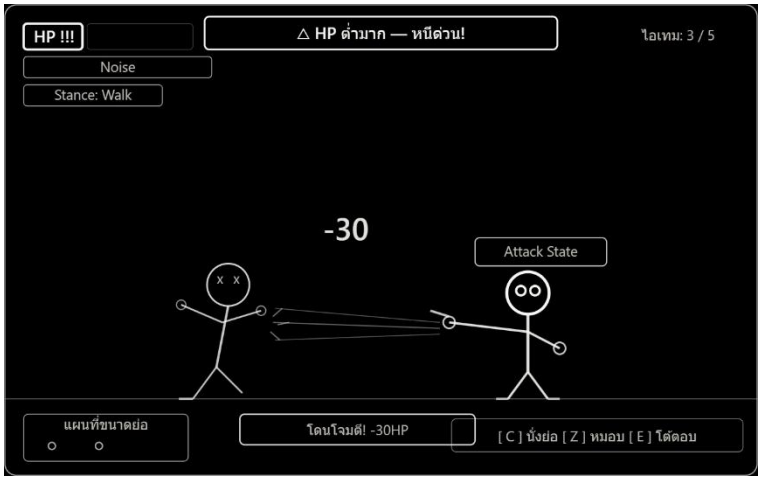
ภาพที่ 3-7 สถานะการโต้ตอบเพื่อเก็บไอเทม

แสดงการทำงานของระบบ Inventory เมื่อตัวละครเข้าใกล้ไอเทมในฉาก ระบบจะตรวจจับการชนระหว่าง Collider ของตัวละครกับไอเทม และแสดง Prompt แจ้งให้ผู้เล่นกดปุ่มโต้ตอบ



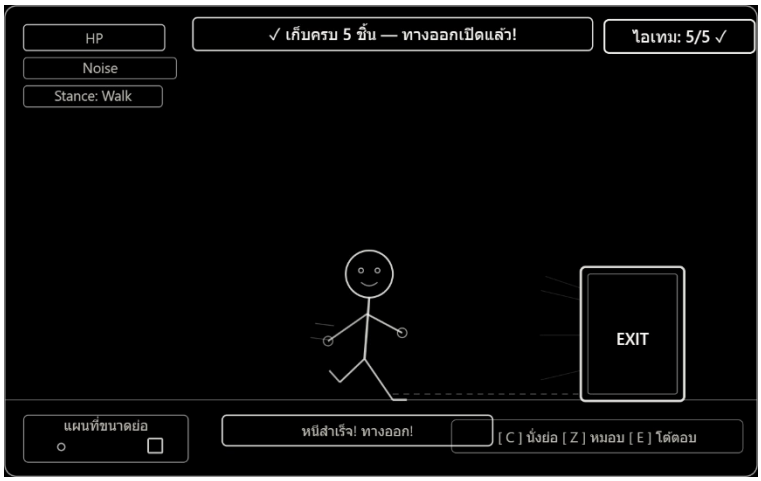
ภาพที่ 3-8 หน้าจอเมื่อตัวละครพบฆาตกร

แสดงการทำงานของระบบ FSM ของศัตรูที่เปลี่ยนเข้าสู่สถานะ Chase State เมื่อตรวจพบผู้เล่นผ่านระบบ Vision Cone โดยศัตรูจะเรียกใช้ A* Algorithm เพื่อคำนวณเส้นทางไล่ล่าแบบ Real-time



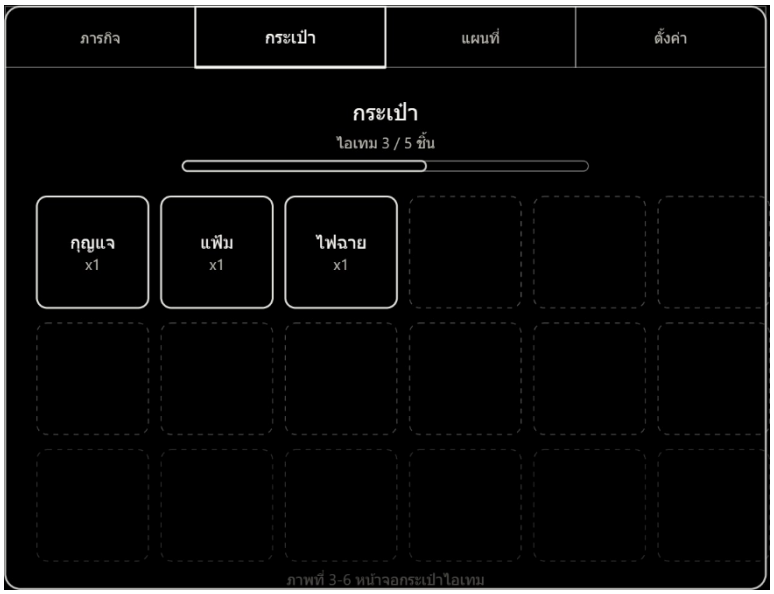
ภาพที่ 3-9 สถานะเมื่อตัวละครได้รับความเสียหายจากการโจมตี

แสดงการทำงานของระบบ FSM ของศัตรูในสถานะ Attack State เมื่อระยะห่างระหว่างศัตรูกับผู้เล่นอยู่ในเกณฑ์ที่กำหนด ระบบจะหักค่า HP ของผู้เล่นและแสดงตัวเลขความเสียหายบนหน้าจอ



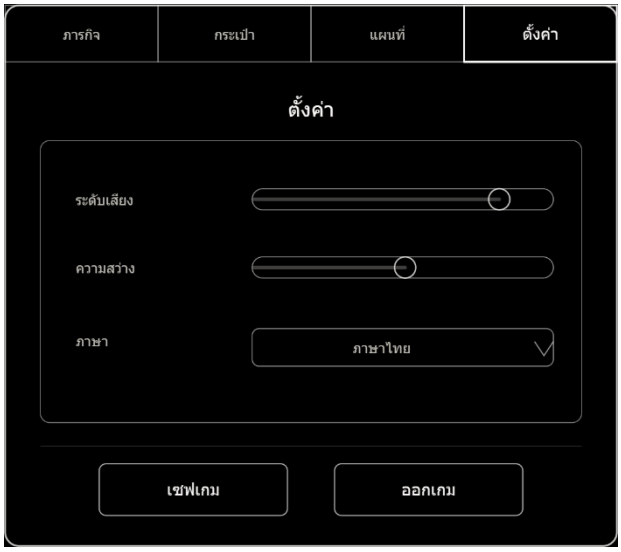
ภาพที่ 3-10 หน้าจอแจ้งเตือนเมื่อหนีสำเร็จ

แสดงผลลัพธ์เมื่อผู้เล่นบรรลุเงื่อนไขของระบบ Inventory Logic ด้วยการเก็บไอเทมครบตามจำนวนที่กำหนด ระบบ Game Logic จะปลดล็อกประตูทางออก (EXIT) และแสดงการแจ้งเตือนผ่าน UI



ภาพที่ 3-11 หน้าจอกระเป๋าไอเทม

หน้าจอกระเป๋าออกแบบให้มีแท็บเมนูด้านบน 4 แท็บ ได้แก่ ภารกิจ กระเป๋า แผนที่ และตั้งค่า โดยแสดง Grid ช่องเก็บไอเทมขนาด 6×3 ช่อง ไอเทมที่เก็บได้จะมีกรอบสีส้มพร้อมแสดงชื่อและจำนวน ส่วนช่องที่ว่างจะแสดงเป็นเส้นประ นอกจากนี้แสดงความคืบหน้าการเก็บไอเทมเทียบกับเป้าหมาย 5 ชิ้นไว้ด้านบน



ภาพที่3-12 หน้าจอตั้งค่า

หน้าจอตั้งค่าใช้แท็บเมนูเดียวกับหน้ากระเป๋า โดยภายในมี Slider สำหรับปรับระดับเสียงและความสว่าง รวมถึง Dropdown สำหรับเลือกภาษา และมีปุ่มเซฟเกมและออกเกมอยู่ด้านล่าง



ภาพที่3-13 หน้าจอ Game Over

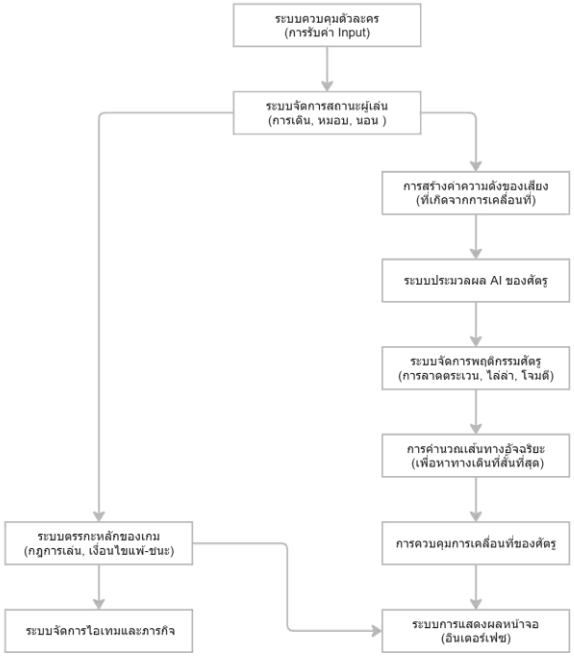
หน้าจอ Game Over ออกแบบให้มีโทนสีเข้มสอดคล้องกับหน้าเมนูหลัก โดยแบ่งออกเป็น 3 ส่วน ได้แก่ ส่วนหัวแสดงชื่อเกม ส่วนกลางแสดงข้อความ Game Over พร้อมเหตุผลและสรุปจำนวนไอเทมที่เก็บได้ และส่วนล่างมีปุ่ม 2 ปุ่ม ได้แก่ Retry สำหรับเริ่มด่านใหม่ และ Exit สำหรับกลับเมนูหลัก

3.5 การพัฒนาระบบ

การพัฒนาระบบเกมในโครงงานนี้ดำเนินการโดยใช้ Unity Game Engine ร่วมกับภาษา C# ตามแบบที่ได้ออกแบบไว้ในขั้นตอนก่อนหน้า โดยมีการออกแบบโครงสร้างระบบในลักษณะเชิงโมดูล (Modular Design) เพื่อให้สามารถพัฒนา แก้ไข และบำรุงรักษาได้อย่างมีประสิทธิภาพ ระบบถูกแบ่งออกเป็น 3 ส่วนหลัก ได้แก่ ระบบควบคุมตัวละครผู้เล่น ระบบปัญญาประดิษฐ์ของศัตรู และระบบตรรกะของเกม ซึ่งแต่ละระบบมีการทำงานร่วมกันอย่างเป็นลำดับ

3.5.1 โครงสร้างสถาปัตยกรรมของระบบ (System Architecture)

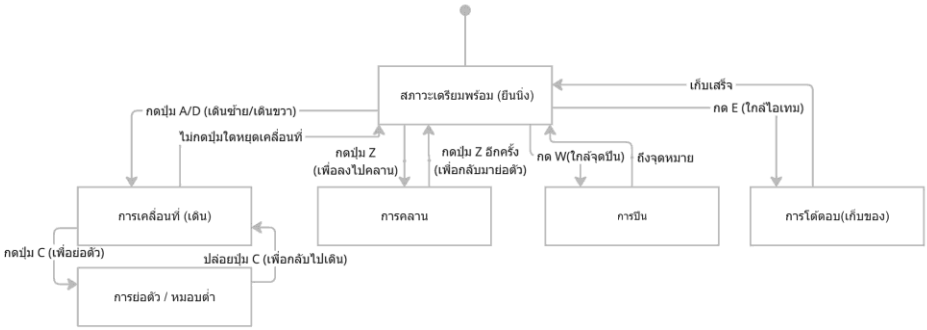
ระบบถูกออกแบบให้มีการทำงานเชื่อมโยงกันระหว่างโมดูลต่าง ๆ โดยระบบควบคุมตัวละครผู้เล่นจะทำหน้าที่รับข้อมูลจากผู้ใช้งาน เช่น การกดปุ่ม และแปลงเป็นพฤติกรรมของตัวละคร ข้อมูลที่เกิดขึ้น เช่น ตำแหน่งของผู้เล่น และระดับเสียงจากการเคลื่อนไหว จะถูกส่งต่อไปยังระบบปัญญาประดิษฐ์ของศัตรู เพื่อใช้ในการประมวลผลการตรวจจับและตัดสินใจเปลี่ยนสถานะ ในขณะเดียวกัน ระบบตรรกะของเกมจะควบคุมเงื่อนไขต่าง ๆ เช่น การเก็บไอเทม การปลดล็อกด่าน และการเปลี่ยนฉาก โดยข้อมูลทั้งหมดจะถูกแสดงผลผ่านระบบส่วนติดต่อผู้ใช้ (UI) แบบเรียลไทม์ โครงสร้างดังกล่าวช่วยให้ระบบมีความยืดหยุ่น สามารถแยกพัฒนาแต่ละส่วนได้อย่างอิสระ และรองรับการขยายระบบในอนาคต



ภาพที่ 3-14 System Integration Diagram

3.5.2 การพัฒนาระบบควบคุมตัวละครผู้เล่น (Player Controller System)

การพัฒนาระบบควบคุมตัวละครผู้เล่นใช้เทคนิค Finite State Machine (FSM) เป็นโครงสร้างหลักในการจัดการสถานะ โดยใช้ State Pattern ในการออกแบบเพื่อให้สามารถจัดการพฤติกรรมได้อย่างเป็นระบบ



ภาพที่ 3-15 FSM Player

3.5.2.1 State Management

ระบบประกอบด้วยสถานะหลัก 4 สถานะ ได้แก่ Idle, Walk, Crouch และ Prone โดยแต่ละสถานะมีเงื่อนไขการเปลี่ยนสถานะที่ชัดเจน เช่น การกดปุ่มควบคุมของผู้เล่น การใช้ FSM ช่วยให้สามารถควบคุมพฤติกรรมของตัวละครได้อย่างเป็นลำดับ ลดความซับซ้อนของโค้ด และเพิ่มความสามารถในการปรับปรุงในอนาคต

3.5.2.2 Dynamic Collider Adjustment

ระบบมีการปรับขนาด Collider ของตัวละครแบบเรียลไทม์ตามสถานะการเคลื่อนไหว เพื่อให้สอดคล้องกับรูปแบบการเคลื่อนที่ในแต่ละสถานะ การปรับขนาดดังกล่าวช่วยให้ตัวละครสามารถเคลื่อนที่ผ่านพื้นที่ที่มีข้อจำกัดด้านขนาดได้อย่างเหมาะสม และเพิ่มความสมจริงให้กับระบบฟิสิกส์ของเกม

3.5.2.3 Variable Parameters

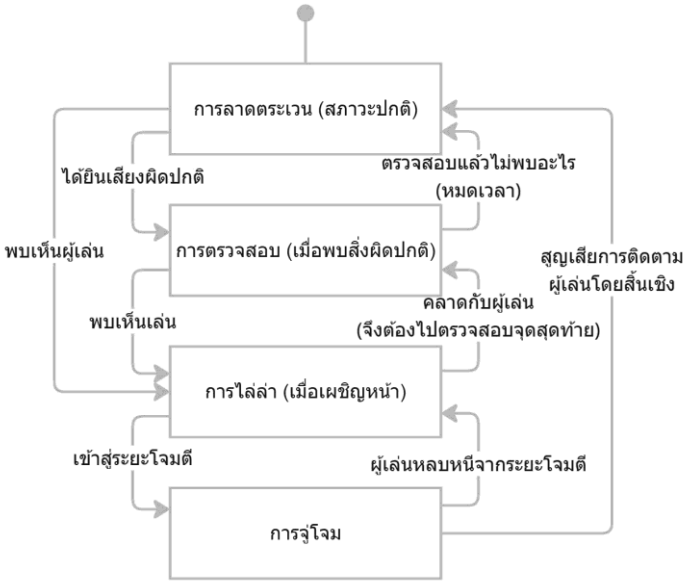
ในแต่ละสถานะมีการกำหนดค่าพารามิเตอร์ที่แตกต่างกัน เช่น ความเร็วในการเคลื่อนที่ และรัศมีเสียง ค่าพารามิเตอร์เหล่านี้ถูกออกแบบให้สัมพันธ์กับรูปแบบการเล่น โดยสถานะที่เคลื่อนที่เร็วจะสร้างเสียงมากกว่า ซึ่งส่งผลต่อโอกาสในการถูกตรวจจับโดยศัตรู การกำหนดลักษณะดังกล่าวช่วยเพิ่มมิติของกลยุทธ์ในการเล่น และส่งเสริมประสบการณ์ของผู้ใช้งาน

3.5.3 การพัฒนาระบบปัญญาประดิษฐ์ของศัตรู (Enemy AI System)

ระบบ AI ของศัตรูถูกออกแบบโดยใช้ FSM ร่วมกับระบบการรับรู้ (Sensory System) และระบบค้นหาเส้นทาง เพื่อให้สามารถตอบสนองต่อผู้เล่นได้อย่างสมจริง

3.5.3.1 FSM ของศัตรู

ศัตรูมีสถานะหลัก ได้แก่ Patrol, Investigate, Chase และ Attack โดยการเปลี่ยนสถานะจะขึ้นอยู่กับข้อมูลที่ได้รับจากระบบการรับรู้ FSM ทำหน้าที่เป็นกลไกหลักในการควบคุมพฤติกรรม ทำให้การตัดสินใจของศัตรูมีโครงสร้างที่ชัดเจน



ภาพที่3-16 FSM Enemy

3.5.3.2 Sensory System

ระบบการรับรู้แบ่งออกเป็นการมองเห็นและการได้ยิน โดยใช้ Raycasting สำหรับการตรวจจับการมองเห็น และการคำนวณระยะทางสำหรับการตรวจจับเสียง ข้อมูลจากระบบนี้จะถูกใช้เป็นเงื่อนไขในการเปลี่ยนสถานะของ FSM เช่น การเข้าสู่สถานะ Investigate หรือ Chase

3.5.3.3 A* Pathfinding

ระบบค้นหาเส้นทางใช้ A* Algorithm ในการคำนวณเส้นทางที่เหมาะสม โดยใช้ฟังก์ชัน $f(n) = g(n) + h(n)$ เพื่อหาค่าเส้นทางที่มีต้นทุนต่ำที่สุด ซึ่งช่วยให้ศัตรูสามารถเคลื่อนที่หลบหลีกสิ่งกีดขวางและไล่ล่าผู้เล่นได้อย่างมีประสิทธิภาพ

3.5.4 การพัฒนาระบบเกม (Game Logic System)

ระบบเกมทำหน้าที่ควบคุมกลไกหลักของเกมและการโต้ตอบกับผู้ใช้งาน

3.5.4.1 Inventory และ Mission Logic

ระบบ Inventory ใช้โครงสร้างข้อมูลแบบ List ในการจัดเก็บไอเทม และมีการตรวจสอบเงื่อนไขของภารกิจอย่างต่อเนื่อง เมื่อผู้เล่นบรรลุเงื่อนไขที่กำหนด ระบบจะดำเนินการเปลี่ยนสถานะของเกม เช่น การปลดล็อกทางออก

3.5.4.2 Scene Management

ระบบจัดการฉากรองรับการโหลดฉากและการส่งต่อข้อมูล โดยใช้รูปแบบ JSON ในการจัดเก็บข้อมูล ช่วยให้สามารถรักษาความต่อเนื่องของเกมได้แม้มีการเปลี่ยนฉาก

3.5.4.3 UI/UX System

ระบบ UI แสดงข้อมูลสำคัญ เช่น สถานะตัวละคร ระดับเสียง จำนวนไอเทม และพลังชีวิต การแสดงผลแบบเรียลไทม์ช่วยให้ผู้เล่นสามารถตัดสินใจได้อย่างมีประสิทธิภาพ

3.5.5 การทำงานร่วมกันของระบบ (System Integration)

ระบบทั้งหมดถูกออกแบบให้ทำงานร่วมกันอย่างเป็นลำดับ โดยข้อมูลจากผู้เล่นจะถูกส่งไปยังระบบ AI เพื่อใช้ในการตัดสินใจ และผลลัพธ์จาก AI จะส่งผลต่อสถานะของเกม FSM ทำหน้าที่เป็นแกนกลางในการควบคุมพฤติกรรม ขณะที่ระบบการรับรู้ทำหน้าที่เป็นตัวกำหนดเงื่อนไขในการเปลี่ยนสถานะ การทำงานร่วมกันของระบบทั้งหมดช่วยให้เกมมีความสมจริง และสามารถตอบสนองต่อการกระทำของผู้เล่นได้อย่างมีประสิทธิภาพ

3.5.6 ลำดับการทำงานของระบบ (System Workflow)

ระบบเกมถูกออกแบบให้ทำงานเป็นลำดับขั้นตอน โดยเริ่มจากการรับข้อมูลจากผู้เล่นและดำเนินการประมวลผลผ่านระบบต่าง ๆ ดังนี้ เมื่อผู้เล่นทำการกดปุ่มควบคุม ระบบจะทำการอัปเดตสถานะของตัวละครผ่าน Finite State Machine (FSM) จากนั้นจะคำนวณค่าพารามิเตอร์ เช่น ความเร็วและระดับเสียงที่เกิดขึ้นจากการเคลื่อนไหว ข้อมูลดังกล่าวจะถูกส่งไปยังระบบปัญญาประดิษฐ์ของศัตรู ซึ่งจะทำการตรวจจับผู้เล่นผ่านระบบการมองเห็นและการได้ยิน หากตรวจพบผู้เล่น ระบบจะทำการเปลี่ยนสถานะของศัตรู เช่น จาก Patrol ไปเป็น Chase หลังจากนั้น ระบบจะคำนวณเส้นทางการเคลื่อนที่ของศัตรูด้วยอัลกอริทึม A* และดำเนินการเคลื่อนที่หรือโจมตีผู้เล่นตามสถานการณ์ สุดท้าย ระบบจะทำการอัปเดตข้อมูลทั้งหมดและแสดงผลผ่านส่วนติดต่อผู้ใช้ (UI) แบบเรียลไทม์

3.5.7 ข้อมูลนำเข้าและผลลัพธ์ของระบบ (Input / Output)

ระบบเกมมีการรับข้อมูลนำเข้าและสร้างผลลัพธ์ ดังนี้

ข้อมูลนำเข้า (Input)

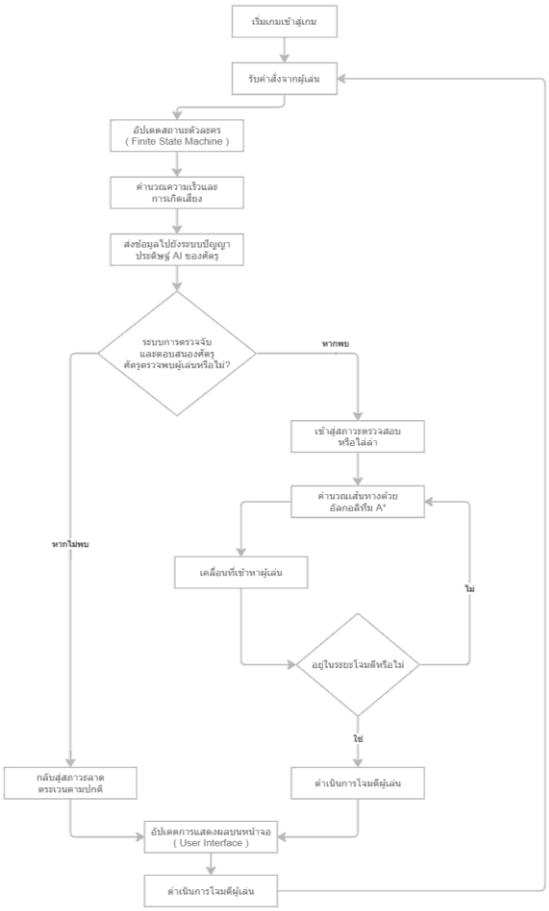
1. การกดปุ่มจากผู้เล่น (Keyboard Input)
2. ตำแหน่งของตัวละคร
3. ระยะห่างระหว่างผู้เล่นและศัตรู

4. สถานะของตัวละคร (State)

ผลลัพธ์ (Output)

1. การเคลื่อนที่ของตัวละคร
2. การเปลี่ยนสถานะของ FSM
3. พฤติกรรมของศัตรู
4. การแสดงผล UI เช่น HP, Item, Noise

การกำหนด Input และ Output อย่างชัดเจนช่วยให้ระบบมีโครงสร้างที่เข้าใจง่าย และสามารถตรวจสอบการทำงานได้อย่างเป็นระบบ



ภาพที่3-17 Flowchart ระบบเกม

3.5.8 การออกแบบเงื่อนไขพิเศษ (Edge Case Handling)

เพื่อให้ระบบมีความสมบูรณ์ ผู้พัฒนาได้กำหนดเงื่อนไขพิเศษสำหรับรองรับสถานการณ์ที่อาจเกิดขึ้น เช่น

1. หากศัตรูไม่พบผู้เล่นภายในระยะเวลาที่กำหนด จะกลับสู่สถานะ Patrol
2. หากผู้เล่นออกนอกระยะตรวจจับ ศัตรูจะหยุดการไล่ล่า
3. หากผู้เล่นอยู่ในสถานะ Prone ศัตรูจะตรวจจับได้ยากขึ้น
4. หากไม่มีเส้นทางที่สามารถเดินได้ ระบบจะหยุดการเคลื่อนที่ของศัตรู

การออกแบบเงื่อนไขดังกล่าวช่วยเพิ่มความเสถียรและความสมจริงของระบบ

3.6 การทดสอบ

ในการพัฒนาเกมนี้ ผู้พัฒนาได้เลือกใช้วิธีการทดสอบแบบ Unit Testing ซึ่งเป็นการทดสอบซอฟต์แวร์ในระดับหน่วยย่อยของระบบ เช่น ฟังก์ชัน เมธอด หรือคลาส โดยทำการแยกทดสอบแต่ละส่วนออกจากระบบหลัก เพื่อให้มั่นใจว่าส่วนประกอบแต่ละส่วนสามารถทำงานได้อย่างถูกต้องตามที่ออกแบบไว้ การทดสอบในลักษณะนี้ช่วยลดความซับซ้อนของการตรวจสอบระบบ เนื่องจากสามารถระบุข้อผิดพลาดได้อย่างชัดเจนในแต่ละส่วน และสามารถแก้ไขได้อย่างรวดเร็ว ส่งผลให้ระบบโดยรวมมีความเสถียรและมีประสิทธิภาพมากยิ่งขึ้น การทดสอบระบบในโครงงานนี้สามารถแบ่งออกเป็นขั้นตอนดังต่อไปนี้

3.6.1 การทดสอบโดยผู้พัฒนา (Developer Testing)

การทดสอบโดยผู้พัฒนาเป็นขั้นตอนเริ่มต้นที่สำคัญ โดยผู้พัฒนาจะทำการตรวจสอบการทำงานของระบบด้วยตนเองในระหว่างและหลังการพัฒนา เพื่อค้นหาข้อผิดพลาดที่อาจเกิดขึ้นในโค้ดหรือโครงสร้างของระบบ ในการทดสอบ ผู้พัฒนาได้สร้างชุดข้อมูลสำหรับทดสอบ (Test Data) ขึ้นมาเพื่อจำลองสถานการณ์ต่าง ๆ ภายในเกม เช่น การเคลื่อนที่ของตัวละคร การตรวจจับผู้เล่นของศัตรู และการโต้ตอบกับสภาพแวดล้อม โดยข้อมูลที่ใช้จะประกอบไปด้วยข้อมูลที่ถูกต้อง ข้อมูลที่ไม่ถูกต้อง และข้อมูลที่เป็นค่าว่าง เพื่อทดสอบความสามารถของระบบในการรองรับสถานการณ์ที่หลากหลาย ผลจากการทดสอบในขั้นตอนนี้ช่วยให้สามารถตรวจพบข้อผิดพลาด เช่น การทำงานผิดเงื่อนไข การคำนวณที่ไม่ถูกต้อง หรือการแสดงผลที่คลาดเคลื่อน และสามารถปรับปรุงแก้ไขได้ตั้งแต่ในระดับต้นของการพัฒนา

3.6.2 การทดสอบระบบ Finite State Machine (FSM)

การทดสอบในส่วนของ FSM มีความสำคัญอย่างมาก เนื่องจากเป็นโครงสร้างหลักที่ใช้ควบคุมพฤติกรรมของตัวละครและศัตรูภายในเกม ผู้พัฒนาได้ทำการตรวจสอบการเปลี่ยนสถานะของระบบ เช่น การเปลี่ยนจากสถานะหยุดนิ่งไปเป็นการเคลื่อนที่ หรือจากการลาดตระเวนไปเป็นการไล่ล่า โดยพิจารณาว่าการเปลี่ยนสถานะนั้นเกิดขึ้นตามเงื่อนไขที่กำหนดไว้หรือไม่ นอกจากนี้ ยังมีการตรวจสอบการทำงานภายในแต่ละสถานะ เช่น การทำงานเมื่อเข้าสู่สถานะ การทำงานระหว่างอยู่ในสถานะ และการทำงานเมื่อออกจากสถานะ เพื่อให้มั่นใจว่าการทำงานของระบบมีความต่อเนื่องและไม่เกิดความผิดพลาด การทดสอบในส่วนนี้ช่วยให้ระบบสามารถควบคุมพฤติกรรมของตัวละครได้อย่างถูกต้อง มีลำดับขั้นตอนที่ชัดเจน และสามารถตอบสนองต่อเหตุการณ์ต่าง ๆ ภายในเกมได้อย่างเหมาะสม

3.6.3 การทดสอบระบบปัญญาประดิษฐ์ของศัตรู (Enemy AI)

การทดสอบระบบปัญญาประดิษฐ์ของศัตรูมุ่งเน้นไปที่การตรวจสอบพฤติกรรมของศัตรูในสถานการณ์ต่าง ๆ เพื่อให้แน่ใจว่าสามารถตอบสนองต่อผู้เล่นได้อย่างสมจริง ผู้พัฒนาได้ทำการจำลองสถานการณ์ เช่น การที่ศัตรูตรวจพบผู้เล่น การไล่ล่า การโจมตี และการกลับไปยังสถานะปกติเมื่อไม่พบผู้เล่น โดยตรวจสอบว่าการตอบสนองของศัตรูมีความสอดคล้องกับเงื่อนไขที่กำหนดไว้หรือไม่ การทดสอบนี้ช่วยให้สามารถประเมินความสมจริงของระบบ AI และปรับปรุงพฤติกรรมของศัตรูให้มีความเหมาะสมกับรูปแบบของเกมมากยิ่งขึ้น

3.6.4 การทดสอบระบบการเคลื่อนที่และ Pathfinding

การทดสอบระบบการเคลื่อนที่มีวัตถุประสงค์เพื่อตรวจสอบความถูกต้องของการเคลื่อนที่ของตัวละครทั้งผู้เล่นและศัตรู ในส่วนของศัตรู ได้มีการทดสอบการเคลื่อนที่โดยใช้ระบบค้นหาเส้นทาง เพื่อให้สามารถเดินทางไปยังตำแหน่งของผู้เล่นได้อย่างเหมาะสม และสามารถหลบหลีกสิ่งกีดขวางภายในฉากได้ การทดสอบในขั้นตอนนี้ช่วยให้มั่นใจว่าการเคลื่อนที่ของตัวละครมีความลื่นไหล ไม่ติดขัด และสามารถทำงานได้อย่างถูกต้องในสภาพแวดล้อมที่หลากหลาย

3.6.5 การทดสอบโดยผู้ใช้งาน (User Testing)

การทดสอบโดยผู้ใช้งานเป็นขั้นตอนที่ใช้ประเมินประสิทธิภาพของระบบจากมุมมองของผู้เล่นจริง โดยให้กลุ่มผู้ใช้งานทดลองเล่นเกม และทำการประเมินผ่านแบบสอบถาม ซึ่งเป็นเครื่องมือในการเก็บรวบรวมข้อมูลความคิดเห็นและความพึงพอใจของผู้ใช้งาน การประเมินแบ่งออกเป็น 2 ด้าน ได้แก่

3.6.5.1 ด้านการทำงานของระบบ เช่น ความถูกต้อง ความเสถียร และการตอบสนองของเกม

3.6.5.2 ด้านการออกแบบและความเป็นมิตรต่อผู้ใช้ เช่น ความง่ายในการใช้งาน ความเข้าใจง่ายของหน้าจอ และความสวยงามของเกม

ผลการประเมินใช้มาตราส่วนประมาณค่า (Rating Scale) 5 ระดับ ดังนี้

ระดับคะแนน	**ความหมาย**
ระดับ 5	ดีมาก
ระดับ 4	ดี
ระดับ 3	ปานกลาง
ระดับ 2	น้อย
ระดับ 1	น้อยมาก

ข้อมูลที่ได้จากการทดสอบโดยผู้ใช้งานจะช่วยสะท้อนถึงประสบการณ์จริงของผู้เล่น และสามารถนำไปใช้ในการปรับปรุงระบบให้มีประสิทธิภาพและตอบสนองต่อความต้องการของผู้ใช้งานได้ดียิ่งขึ้น

3.6.6 การวิเคราะห์ผลการทดสอบ

ข้อมูลที่ได้จากแบบประเมินจะถูกนำมาวิเคราะห์โดยใช้สถิติพื้นฐาน เพื่อสรุประดับประสิทธิภาพของระบบ โดยใช้ค่าเฉลี่ยและค่าส่วนเบี่ยงเบนมาตรฐานเป็นตัวชี้วัด

สูตรการหาค่าเฉลี่ย

$$\bar{x} = \frac{\sum x}{n}$$

เมื่อ

$\bar{x}$ แทน ค่าเฉลี่ย

$\sum x$ แทน ผลรวมของคะแนนทั้งหมด

n แทน จำนวนข้อมูล

สูตรการหาส่วนเบี่ยงเบนมาตรฐาน

$$S.D. = \sqrt{\frac{\Sigma(\mathrm{x} - \bar{\mathrm{x}})^2}{n-1}}$$

เมื่อ

$S.D.$ แทนค่า ส่วนเบี่ยงเบนมาตรฐาน

X แทนค่า ค่าคะแนนแต่ละตัว

$\bar{\mathrm{x}}$ แทนค่า ค่าเฉลี่ย

n แทน จำนวนข้อมูล

เกณฑ์การแปลผลค่าเฉลี่ย

ช่วงคะแนน	**ระดับ**
4.51 – 5.00	ดีมาก
3.51 – 4.50	ดี
2.51 – 3.50	ปานกลาง
1.51 – 2.50	น้อย
1.00 – 1.50	น้อยมาก

3.7 การบำรุงรักษา

การบำรุงรักษาระบบเป็นขั้นตอนสำคัญในวงจรการพัฒนาซอฟต์แวร์ ซึ่งดำเนินการภายหลังจากที่ระบบได้ถูกพัฒนาและนำไปใช้งานจริง โดยมีวัตถุประสงค์เพื่อแก้ไขข้อผิดพลาด ปรับปรุงประสิทธิภาพ และพัฒนาระบบให้สามารถรองรับการเปลี่ยนแปลงได้อย่างเหมาะสม ในการพัฒนาเกมนี้ ผู้พัฒนาได้ดำเนินการบำรุงรักษาระบบโดยอ้างอิงตามประเภทของการบำรุงรักษาซอฟต์แวร์มาตรฐาน ซึ่งสามารถแบ่งออกได้เป็น 4 ประเภท ได้แก่ การบำรุงรักษาเพื่อแก้ไขข้อผิดพลาด การบำรุงรักษาเพื่อปรับตัว การบำรุงรักษาเพื่อพัฒนาเพิ่มเติม และการบำรุงรักษาเชิงป้องกัน โดยมีรายละเอียดดังนี้

3.7.1 การบำรุงรักษาเพื่อแก้ไขข้อผิดพลาด (Corrective Maintenance)

การบำรุงรักษาประเภทนี้เป็นการแก้ไขข้อผิดพลาดที่เกิดขึ้นหลังจากระบบถูกนำไปใช้งานจริง ซึ่งข้อผิดพลาดอาจเกิดจากความบกพร่องของโปรแกรม การออกแบบที่ไม่สมบูรณ์ หรือเงื่อนไขที่ไม่ได้คาดการณ์ไว้ในระหว่างการพัฒนา สำหรับโครงงานนี้ การบำรุงรักษาเพื่อแก้ไขข้อผิดพลาดมุ่งเน้นไปที่การตรวจสอบและแก้ไขการทำงานของระบบหลัก เช่น ระบบ Finite State Machine (FSM) และระบบปัญญาประดิษฐ์ของศัตรู โดยเฉพาะในกรณีที่พบว่าการเปลี่ยนสถานะไม่เป็นไปตามเงื่อนไข หรือพฤติกรรมของศัตรูไม่สอดคล้องกับสถานการณ์ภายในเกม การดำเนินการในลักษณะนี้ช่วยให้ระบบมีความถูกต้องและสามารถทำงานได้ตามที่ออกแบบไว้

3.7.2 การบำรุงรักษาเพื่อปรับตัว (Adaptive Maintenance)

การบำรุงรักษาประเภทนี้เป็นการปรับปรุงระบบเพื่อให้สามารถรองรับการเปลี่ยนแปลงของสภาพแวดล้อม เช่น การเปลี่ยนแปลงของฮาร์ดแวร์ ซอฟต์แวร์ ระบบปฏิบัติการ หรือข้อกำหนดในการใช้งาน ในโครงงานนี้ การบำรุงรักษาเพื่อปรับตัวถูกนำมาใช้ในการปรับปรุงประสิทธิภาพของเกมให้สามารถทำงานได้บนอุปกรณ์ที่มีความหลากหลาย รวมถึงการปรับค่าพารามิเตอร์ของระบบ AI และ FSM เพื่อให้เหมาะสมกับรูปแบบการเล่นหรือระดับความยากของเกม การดำเนินการดังกล่าวช่วยให้ระบบสามารถทำงานได้อย่างมีประสิทธิภาพในสภาพแวดล้อมที่แตกต่างกัน

3.7.3 การบำรุงรักษาเพื่อพัฒนาเพิ่มเติม (Perfective Maintenance)

การบำรุงรักษาประเภทนี้เป็นการปรับปรุงระบบเพื่อเพิ่มประสิทธิภาพหรือเพิ่มความสามารถใหม่ โดยไม่ได้เกิดจากข้อผิดพลาด แต่เป็นการพัฒนาเพื่อให้ระบบตอบสนองต่อความต้องการของผู้ใช้งานได้ดียิ่งขึ้น ในโครงงานนี้ การบำรุงรักษาเพื่อพัฒนาเพิ่มเติมครอบคลุมถึงการเพิ่มฟังก์ชันใหม่ เช่น การเพิ่มสถานะในระบบ FSM เพื่อให้พฤติกรรมของศัตรูมีความหลากหลายมากขึ้น รวมถึงการปรับปรุงระบบปัญญาประดิษฐ์ให้มีความสมจริง และการพัฒนาส่วนติดต่อผู้ใช้ให้มีความสะดวกต่อการใช้งาน การดำเนินการในส่วนนี้ช่วยเพิ่มคุณภาพของระบบและยกระดับประสบการณ์ของผู้เล่น

3.7.4 การบำรุงรักษาเชิงป้องกัน (Preventive Maintenance)

การบำรุงรักษาประเภทนี้เป็นการดำเนินการเพื่อป้องกันปัญหาที่อาจเกิดขึ้นในอนาคต โดยเน้นการปรับปรุงโครงสร้างของระบบให้มีความเสถียรและง่ายต่อการดูแลรักษา สำหรับโครงงานนี้ ผู้พัฒนาได้ดำเนินการปรับปรุงโค้ดให้มีโครงสร้างที่ชัดเจนและเป็นระเบียบ เช่น การแยกโมดูลของระบบ FSM และ AI อย่างเหมาะสม การเขียนคำอธิบายประกอบโค้ด และการจัดการเวอร์ชันของระบบ การบำรุงรักษาในลักษณะนี้ช่วยลดความเสี่ยงของการเกิดข้อผิดพลาดในอนาคต และช่วยให้สามารถพัฒนาระบบต่อได้อย่างมีประสิทธิภาพ